

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG DỮ LIỆU



BÁO CÁO ĐỒ ÁN
TaskOps Cloud Platform

GVHD: Lê Anh Tuấn

Nhóm sinh viên thực hiện:

- | | |
|-------------------|----------------|
| 1. Nguyễn Đức Thọ | MSSV: 23521518 |
| 2. Đoàn Thanh Độ | MSSV: 23520282 |
| 3. Đoàn Văn Tài | MSSV: 23521371 |

Mục Lục

Danh mục Hình ảnh	3
Danh mục Bảng	6
Chương 1: GIỚI THIỆU ĐỀ TÀI.....	7
1.1Bối cảnh và lý do chọn đề tài.....	7
1.2Mục tiêu của đề tài.....	7
1.3Phạm vi thực hiện	8
1.4Ý nghĩa thực tiễn của đề tài	9
Chương 2. CƠ SỞ LÝ THUYẾT	10
2.1Tổng quan về DevOps	10
2.2CI/CD	10
2.3Infrastructure as Code	12
2.4GitOps	13
2.5AWS EKS	14
2.6Các công nghệ DevOps sử dụng.....	15
Chương 3. TRIỂN KHAI.....	22
3.1Kiến trúc CI/CD pipeline	22
3.2Kiến trúc hệ thống AWS.....	35
3.3Xây dựng hạ tầng AWS bằng Terraform.....	38
3.4Triển khai Kubernetes bằng GitOps	51
3.5Task Manager Helm chart.....	59
3.6Quy trình Rollout backend.....	62
3.7Tự động hóa vận hành.....	65
Chương 4. KẾT QUẢ & KẾT LUẬN	78
4.1Kết quả đạt được	78
4.2Đánh giá hệ thống	78
4.3Hạn chế	79
4.4Hướng phát triển	79
4.5Kết luận	79
TÀI LIỆU THAM KHẢO	81

Danh mục Hình ảnh

Hình 1. Mô hình Devops truyền thống.....	10
Hình 2 Kiến trúc tổng quát của một CI/CD pipeline.....	12
Hình 3. Workflow và các thành phần của Terraform.....	13
Hình 4. Mô hình GitOps.....	14
Hình 5. Github Actions.....	15
Hình 6. HashiCorp Terraform	16
Hình 7. Helm	16
Hình 8. Kustomize.....	17
Hình 9. ArgoCD	17
Hình 10. Argo Rollouts	18
Hình 11. External Secrets Operator.....	18
Hình 12. Prometheus & Grafana	19
Hình 13. Trivy	20
Hình 14. SonarQube	20
Hình 15. Kiến trúc CI/CD pipeline.....	22
Hình 16. Các trường hợp trigger CI workflow	23
Hình 17. Cấu hình concurrency cho workflow	23
Hình 18. Các stage trong CI pipeline	24
Hình 19. Backend lint and test	24
Hình 20. Frontend lint and test.....	25
Hình 21. Cấu hình sonar-project.properties cho SonarQube.....	25
Hình 22. Source code quy trình kiểm thử backend/frontend	26
Hình 23. Source code quy trình scan file system	28
Hình 24. Source code quy trình scan SAST SonarQube.....	29
Hình 25. Source code scan secrets	30
Hình 26. Source code quy trình scan Docker image backend local	31
Hình 27. Source code quy trình update config repo.....	34
Hình 28. Kiến trúc hệ thống AWS	35
Hình 29. Mô hình Terraform workflow	38
Hình 30. Cấu trúc bootstrap remote state.....	38
Hình 31. Terraform S3 Bucket tfstate và dynamoDB lock state.....	39
Hình 32. Dynamic Tên bucket và lock table	39
Hình 33. Local backend cho bootstrap state.....	39
Hình 34. Các state còn lại dùng S3 Bucket lưu tfstate	40
Hình 35. Cấu trúc network state nằm trong infrastructure/envs/dev.....	40
Hình 36. NAT Gateway, DNS settings, public subnet, private subnet	40
Hình 37. EKS state nằm trong infrastructure/envs/eks	41
Hình 38. cấu hình managed node group cho EKS cluster.....	42
Hình 39. API Endpoint để kubectl từ máy cá nhân.....	42
Hình 40. Cluster addons	43
Hình 41. Cluster secret encryption	43
Hình 42. AWS Load Balancer Controller addons bằng helm	44
Hình 43. External Secrets Operator addons bằng helm.....	44

Hình 44. Cấu trúc RDS state	45
Hình 45. RDS security group	45
Hình 46. RDS KMS encryption.....	46
Hình 47. RDS instance	47
Hình 48. Cấu trúc Secrets state.....	48
Hình 49. Tạo JWT secret trong AWS Secrets Manager.....	48
Hình 50. Tạo Grafana admin password trong AWS Secrets Manager.....	48
Hình 51. Cấu trúc DNS state	49
Hình 52. AWS ACM certificate phục vụ TLS cho ALB	49
Hình 53. Tạo Route53 record và ACM cert validation	50
Hình 54. lookup ALB sau khi tạo ALB.....	50
Hình 55. Tạo Route53 alias record.....	51
Hình 56. Monitoring-extras Application	52
Hình 57. kube-prometheus-stack Application	54
Hình 58. Argo-rollouts Application	56
Hình 59. grafana-dashboards Application.....	57
Hình 60. Task-manager-dev Application	58
Hình 61. Task Manager Helm chart	59
Hình 62. Backend helm values	60
Hình 63. Frontend helm values	60
Hình 64. Backend Rollout helm values.....	61
Hình 65. Secrets helm values	61
Hình 66. Backend rollout aws values	62
Hình 67. metric prometheus-target-up	63
Hình 68. Metric http-success-rate.....	63
Hình 69. metric p99-latency	64
Hình 70. Cấu hình Makefile tập trung.....	65
Hình 71. Lệnh Make preflight	66
Hình 72. Lệnh make sync-config-repo	67
Hình 73. Makefile bootstrap.....	68
Hình 74. Makefile deploy.....	68
Hình 75. Stage 1 makefile infrastructure.....	70
Hình 76. Stage 3 makefile sync helm values.....	71
Hình 77. Stage 4 makefile argocd	71
Hình 78. Makefile install-argocd.....	72
Hình 79. Makefile bootstrap-apps	72
Hình 80. Stage 5 makefile wait-alb	73
Hình 81. Makefile status.....	74
Hình 82. Makefile verify	74
Hình 83. Makefile hibernate.....	75
Hình 84. Makefile wake	76
Hình 85. Makefile argocd-ui	76
Hình 86. Makefile logs-backend, logs-frontend, shell-backend	77

Danh mục Bảng

Bảng 1. Danh sách các Stage trong CI pipeline	23
Bảng 2. Danh sách các ArgoCD Application.....	52
Bảng 3. Các kết quả đạt được trong sau quá trình thực hiện.....	78

Chương 1: GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và lý do chọn đề tài

Trong các hệ thống phần mềm hiện đại, yêu cầu phát hành nhanh, ổn định và an toàn ngày càng trở nên quan trọng. Một ứng dụng web không chỉ cần chạy đúng về mặt chức năng, mà còn cần được triển khai theo quy trình có thể kiểm thử, có khả năng truy vết, có thể rollback khi xảy ra sự cố và có thể mở rộng khi lưu lượng tăng. Nếu triển khai thủ công, nhóm phát triển thường phải thực hiện nhiều thao tác lặp lại như build source code, tạo Docker image, cập nhật cấu hình, apply manifest Kubernetes, kiểm tra log và xử lý sự cố. Các thao tác này dễ phát sinh lỗi, đặc biệt khi hạ tầng có nhiều thành phần như database, secret, ingress, monitoring và controller phụ trợ.

DevOps ra đời để giải quyết bài toán đó bằng cách kết hợp phát triển phần mềm với vận hành hệ thống thông qua tự động hóa, quy trình chuẩn hóa và vòng phản hồi nhanh. Trong đề án này, nhóm lựa chọn xây dựng một ứng dụng Task Manager full-stack và triển khai theo hướng DevOps/GitOps trên AWS EKS. Ứng dụng Task Manager tuy có chức năng nghiệp vụ đơn giản, nhưng đủ để đại diện cho một hệ thống web thực tế: có frontend, backend, xác thực người dùng, cơ sở dữ liệu, health check, metrics và yêu cầu bảo mật. Trên nền ứng dụng đó, nhóm triển khai đầy đủ các lớp DevOps gồm Docker, GitHub Actions, Terraform, Kubernetes, Helm, ArgoCD, Argo Rollouts, External Secrets, Prometheus và Grafana.

Lý do chọn đề tài này là vì nó cho phép nhóm thực hành chuỗi triển khai end-to-end, từ lúc lập trình viên push code lên GitHub đến lúc ứng dụng được cập nhật trên Kubernetes cluster. Thay vì chỉ dừng lại ở việc chạy ứng dụng bằng Docker Compose hoặc apply manifest thủ công, đề án hướng đến một mô hình production-like: CI build và scan image, config repo lưu trạng thái mong muốn, ArgoCD đồng bộ cluster, Argo Rollouts hỗ trợ blue/green deployment, Prometheus/Grafana theo dõi sức khỏe hệ thống và Terraform quản lý hạ tầng AWS bằng mã nguồn.

1.2 Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống web Task Manager và triển khai hệ thống đó bằng quy trình DevOps/GitOps hoàn chỉnh trên AWS EKS. Đề tài không tập trung vào độ phức tạp nghiệp vụ của ứng dụng, mà tập trung vào cách ứng dụng được đóng gói, kiểm thử, triển khai, quan sát và vận hành.

Các mục tiêu cụ thể gồm:

1. Xây dựng ứng dụng Task Manager gồm frontend React/Vite, backend Node.js Express và cơ sở dữ liệu PostgreSQL.
2. Thiết kế backend có xác thực JWT, API quản lý task, health check, logging có cấu trúc và Prometheus metrics.
3. Đóng gói backend và frontend thành Docker image riêng bằng multi-stage build, có runtime tối giản và chạy non-root.

4. Xây dựng pipeline CI/CD bằng GitHub Actions để lint, test, scan bảo mật, build image, push Docker Hub và cập nhật config repo.
5. Dùng Terraform để tạo hạ tầng AWS gồm VPC, EKS, RDS, Secrets Manager, KMS, Route53 và ACM.
6. Tách repository ứng dụng và repository cấu hình GitOps, trong đó config repo là nguồn sự thật cho trạng thái Kubernetes mong muốn.
7. Triển khai ứng dụng lên EKS bằng ArgoCD, Helm chart và values theo môi trường.
8. Áp dụng External Secrets Operator để đồng bộ secret từ AWS Secrets Manager vào Kubernetes mà không lưu secret thật trong Git.
9. Áp dụng Argo Rollouts blue/green deployment để kiểm tra phiên bản backend mới trước khi chuyển traffic.
10. Thiết lập observability bằng Prometheus, ServiceMonitor, PrometheusRule và Grafana RED dashboard.
11. Bổ sung các lớp bảo mật như NetworkPolicy, securityContext, secret scanning, SAST, SCA và IaC scanning.
12. Xây dựng Makefile và script hỗ trợ triển khai, kiểm tra, hibernate/wake và đồng bộ Helm values sau khi tái tạo hạ tầng.

1.3 Phạm vi thực hiện

Phạm vi của đề án gồm hai phần chính: phần ứng dụng và phần nền tảng triển khai. Phần ứng dụng bao gồm Task Manager theo mô hình frontend - backend - database. Frontend được xây dựng bằng React/Vite, backend bằng Node.js Express, Sequelize ORM và PostgreSQL. Ứng dụng hỗ trợ đăng ký, đăng nhập, lưu JWT token, tạo task, xem danh sách task, cập nhật trạng thái và xóa task. Backend có các endpoint health check để phục vụ Kubernetes probe và ALB health check, đồng thời expose endpoint `/metrics` để Prometheus scrape.

Phần nền tảng triển khai bao gồm hạ tầng AWS, Kubernetes và các công cụ DevOps. Hạ tầng AWS được quản lý bằng Terraform theo nhiều state độc lập: network, EKS, RDS, secrets và DNS. Kubernetes chạy trên AWS EKS, sử dụng AWS Load Balancer Controller để tạo ALB từ Ingress, External Secrets Operator để lấy secret từ AWS Secrets Manager, ArgoCD để đồng bộ GitOps, Argo Rollouts để triển khai blue/green và kube-prometheus-stack để cung cấp Prometheus/Grafana.

Trong phạm vi đề án, môi trường chính là môi trường dev trên AWS. Một số cấu hình production-ready đã được chuẩn bị trong Helm chart như HPA, PodDisruptionBudget, values-prod và template security, nhưng không phải tất cả đều bật trong môi trường dev để tiết kiệm tài nguyên. Đề án cũng chưa đi sâu vào các chủ đề như multi-region, disaster recovery đầy đủ, service mesh, chaos engineering hoặc tối ưu chi phí ở quy

mô doanh nghiệp. Tuy nhiên, kiến trúc đã đủ để minh họa quy trình DevOps/GitOps end-to-end và có thể mở rộng thêm trong các giai đoạn sau.

1.4 Ý nghĩa thực tiễn của đề tài

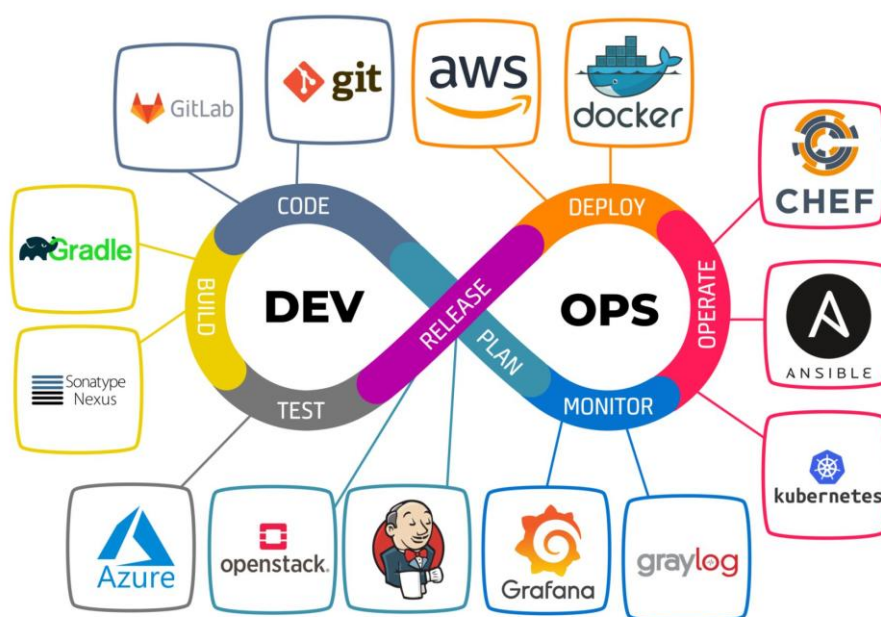
Ý nghĩa chính của đề án nằm ở việc mô phỏng một hệ thống triển khai gần với thực tế, trong đó mỗi thành phần đảm nhận một vai trò rõ ràng. Source code không được deploy trực tiếp lên server, mà được build thành Docker image. Image không được apply vào cluster bằng thao tác thủ công, mà image tag được cập nhật vào config repo. Cluster không nhận lệnh deploy trực tiếp từ CI, mà ArgoCD theo dõi config repo và đồng bộ trạng thái mong muốn. Secret không được commit vào Git, mà được lưu trong AWS Secrets Manager và đồng bộ qua External Secrets Operator. Khi có phiên bản backend mới, Argo Rollouts có thể tạo preview service để kiểm tra trước khi promote.

Mô hình này giúp giảm rủi ro vận hành theo nhiều cách. Thứ nhất, mọi thay đổi hạ tầng và cấu hình đều có thể versioning bằng Git. Thứ hai, pipeline CI/CD có thể chặn lỗi sớm qua test, scan bảo mật và phân tích chất lượng mã nguồn. Thứ ba, GitOps giúp rollback bằng cách revert commit trong config repo. Thứ tư, observability giúp phát hiện lỗi thông qua metrics, dashboard và health check. Cuối cùng, việc dùng Terraform và Makefile giúp tái tạo môi trường nhanh hơn, phù hợp với bối cảnh sinh viên cần tối ưu chi phí và có thể destroy/recreate hạ tầng khi không sử dụng.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về DevOps

DevOps là một phương pháp tổ chức quy trình phát triển và vận hành phần mềm nhằm rút ngắn thời gian đưa sản phẩm ra môi trường sử dụng, đồng thời nâng cao độ ổn định, khả năng kiểm soát và khả năng phản hồi khi có sự cố. DevOps không phải là một công cụ đơn lẻ. Nó là sự kết hợp giữa văn hóa làm việc, quy trình, tự động hóa và bộ công cụ kỹ thuật. Mục tiêu của DevOps là giảm khoảng cách giữa nhóm phát triển phần mềm và nhóm vận hành hệ thống, giúp hai bên cùng chịu trách nhiệm về chất lượng, độ tin cậy và vòng đời của sản phẩm.



Hình 1. Mô hình Devops truyền thống

Trong mô hình truyền thống, phần mềm thường được phát triển xong rồi mới chuyển sang đội vận hành để triển khai. Cách làm này dễ tạo ra độ trễ và sai lệch thông tin: môi trường phát triển khác môi trường triển khai, tài liệu cấu hình không đầy đủ, thao tác thủ công khó lặp lại và lỗi triển khai thường chỉ được phát hiện muộn. DevOps giải quyết các vấn đề đó bằng cách chuẩn hóa quy trình từ lúc viết mã đến lúc vận hành, tự động hóa các bước lặp lại, đưa kiểm thử và bảo mật vào sớm hơn, đồng thời tạo vòng phản hồi liên tục thông qua log, metrics, dashboard và cảnh báo.

Vòng đời DevOps thường được mô tả bằng các giai đoạn liên tục: plan, code, build, test, release, deploy, operate và monitor. Các giai đoạn này không phải là một đường thẳng kết thúc sau khi deploy. Khi hệ thống vận hành, dữ liệu quan sát từ môi trường chạy thật tiếp tục quay lại giai đoạn plan và code để cải thiện phần mềm. Vì vậy, DevOps thường được minh họa như một vòng lặp thay vì một chuỗi công việc một chiều.

2.2 CI/CD

CI/CD là một nhóm thực hành quan trọng trong DevOps. CI là viết tắt của Continuous Integration, nghĩa là tích hợp liên tục. CD có thể là Continuous Delivery hoặc

Continuous Deployment. Continuous Delivery là khả năng luôn giữ phần mềm ở trạng thái sẵn sàng triển khai, còn Continuous Deployment là việc tự động triển khai phiên bản mới khi các điều kiện kiểm thử đạt yêu cầu.

2.2.1 Continuous Integration

Continuous Integration tập trung vào việc tích hợp thay đổi mã nguồn thường xuyên vào repository chung. Mỗi lần có commit hoặc pull request, hệ thống CI sẽ tự động chạy các bước như cài dependency, kiểm tra cú pháp, chạy lint, build và test. Mục tiêu là phát hiện lỗi càng sớm càng tốt. Nếu lỗi được phát hiện ngay sau một commit nhỏ, việc truy vết nguyên nhân và sửa lỗi sẽ dễ hơn nhiều so với việc phát hiện lỗi sau nhiều ngày tích hợp.

Một pipeline CI tốt thường có tính lặp lại cao. Cùng một commit, cùng một pipeline, nếu chạy trong điều kiện giống nhau thì phải tạo ra kết quả giống nhau. Vì vậy, các pipeline hiện đại thường dùng lockfile dependency, container, cache có kiểm soát và môi trường chạy tách biệt để giảm sai lệch.

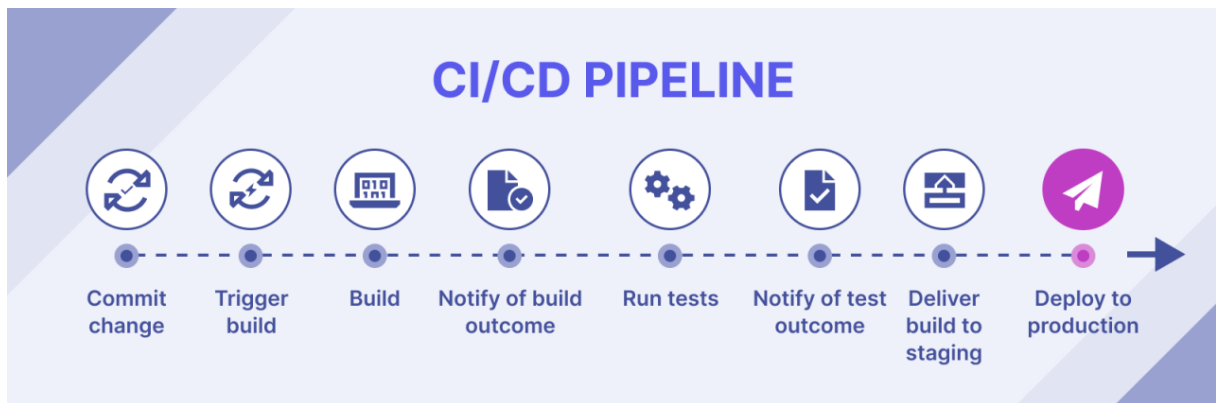
2.2.2 Continuous Delivery và Continuous Deployment

Continuous Delivery mở rộng CI bằng cách chuẩn bị artifact và cấu hình để phiên bản mới có thể được triển khai bất kỳ lúc nào. Artifact có thể là Docker image, package, binary hoặc bundle static files. Continuous Deployment đi xa hơn: nếu các bước kiểm thử và kiểm tra chất lượng đều đạt, hệ thống tự động đưa phiên bản mới lên môi trường chạy thật mà không cần phê duyệt thủ công.

Sự khác biệt chính nằm ở bước ra quyết định cuối cùng. Với Continuous Delivery, con người vẫn quyết định thời điểm deploy. Với Continuous Deployment, quyết định này được tự động hóa dựa trên pipeline. Trong các hệ thống yêu cầu kiểm soát cao, Continuous Delivery thường được ưu tiên hơn vì cho phép có thêm bước review hoặc approval.

2.2.3 Kiến trúc CI/CD Pipeline

Một pipeline CI/CD thường gồm nhiều stage. Stage đầu tiên lấy mã nguồn từ hệ thống quản lý phiên bản. Stage tiếp theo cài dependency, build, chạy test và phân tích mã nguồn. Sau đó pipeline có thể quét bảo mật dependency, quét secret, build container image, scan image, publish artifact và triển khai. Các stage có thể chạy tuần tự hoặc song song tùy quan hệ phụ thuộc.



Hình 2 Kiến trúc tổng quát của một CI/CD pipeline

Điểm quan trọng của pipeline là khả năng chặn lỗi trước khi lỗi đi xa hơn. Nếu test fail, pipeline dừng trước khi build image. Nếu image có lỗi hỏng nghiêm trọng, pipeline có thể dừng trước khi publish hoặc deploy. Nhờ đó CI/CD không chỉ giúp deploy nhanh hơn mà còn đóng vai trò như một lớp kiểm soát chất lượng và bảo mật.

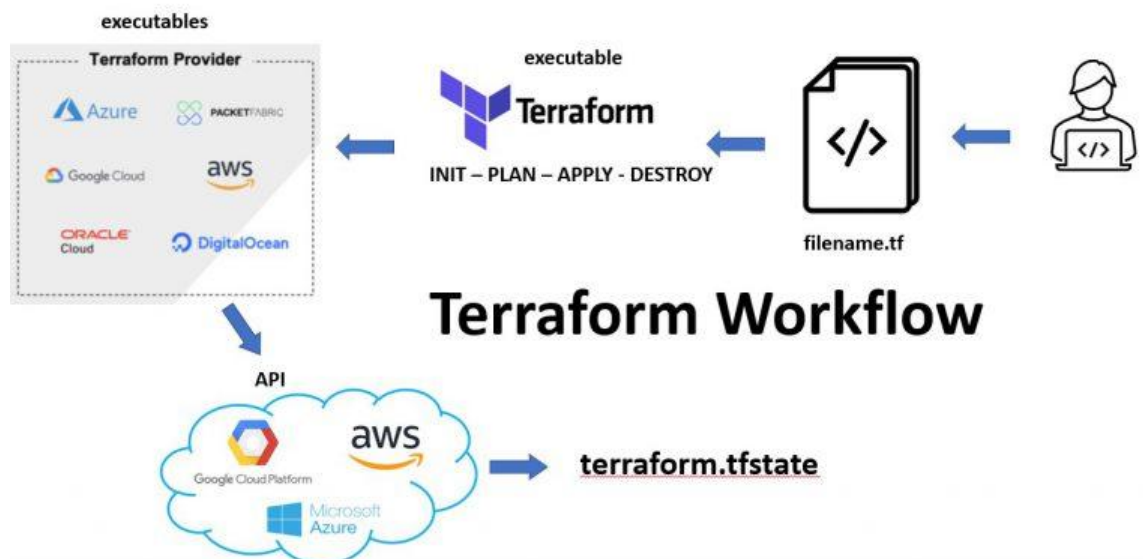
2.3 Infrastructure as Code

Infrastructure as Code là phương pháp mô tả hạ tầng bằng mã nguồn thay vì cấu hình thủ công qua giao diện web. Hạ tầng có thể là network, máy chủ, database, IAM policy, DNS, load balancer, Kubernetes cluster hoặc secret store. Khi hạ tầng được mô tả bằng code, nó có thể được lưu trong Git, review, test, tái tạo và rollback dễ hơn.

Terraform là một công cụ IaC phổ biến. Terraform sử dụng ngôn ngữ HCL để khai báo resource. Các provider như AWS, Azure, Google Cloud hoặc Kubernetes cung cấp schema và API để Terraform quản lý resource tương ứng. Terraform hoạt động theo mô hình declarative: người dùng mô tả trạng thái mong muốn, Terraform tính toán khác biệt giữa trạng thái mong muốn và trạng thái hiện tại, sau đó tạo execution plan và apply.

Kiến trúc và workflow của Terraform

Terraform workflow thường gồm các bước `init`, `plan` và `apply`. `terraform init` tải provider và module. `terraform plan` so sánh configuration với state và resource thật để tạo kế hoạch thay đổi. `terraform apply` thực thi kế hoạch. Terraform state là thành phần rất quan trọng vì nó lưu ánh xạ giữa resource trong code và resource thật. Với môi trường làm việc nhóm, state thường được lưu remote và có cơ chế lock để tránh nhiều người apply cùng lúc.



Hình 3. Workflow và các thành phần của Terraform

Terraform module giúp đóng gói một nhóm resource có liên quan để tái sử dụng. Ví dụ, một module network có thể tạo VPC, subnet, route table và NAT gateway. Một module Kubernetes có thể tạo cluster, node group và addon. Module làm cho cấu hình hạ tầng dễ tổ chức hơn, nhưng cũng cần quản lý version rõ ràng để tránh thay đổi ngoài ý muốn.

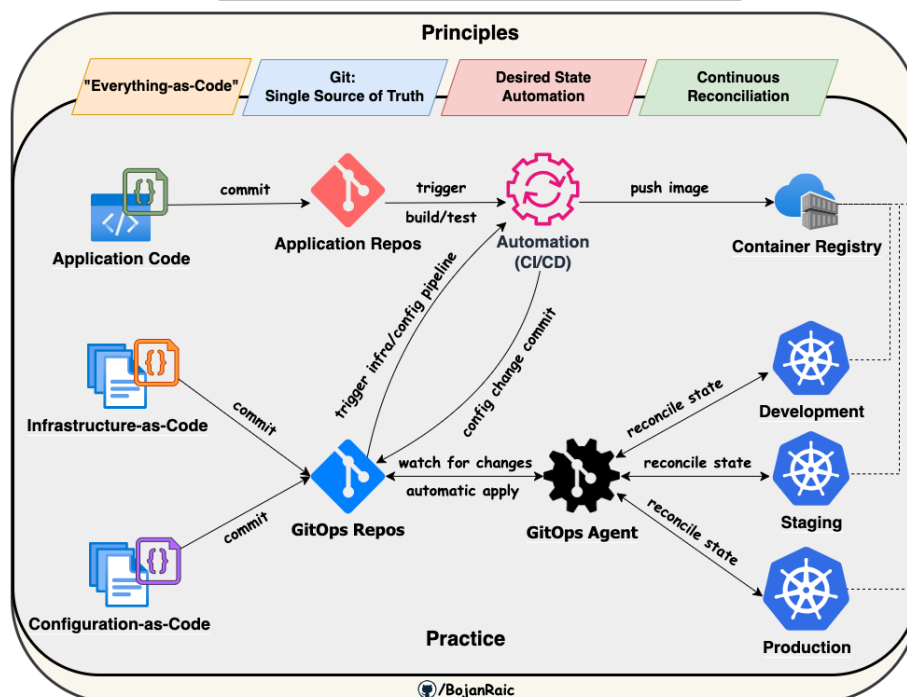
2.4 GitOps

GitOps là một mô hình vận hành trong đó Git repository là nguồn sự thật của trạng thái mong muốn. Các thay đổi cấu hình được thực hiện bằng commit, pull request và merge. Một GitOps controller chạy trong môi trường triển khai sẽ theo dõi repository, so sánh trạng thái thật với trạng thái mong muốn và đồng bộ khi có khác biệt. GitOps giúp tăng tính minh bạch vì mọi thay đổi đều có lịch sử commit, người thực hiện, thời điểm và nội dung cụ thể.

Nguyên lý GitOps

GitOps thường dựa trên bốn nguyên lý. Thứ nhất, toàn bộ trạng thái mong muốn được khai báo theo cách declarative. Thứ hai, trạng thái mong muốn được lưu trong hệ thống quản lý phiên bản. Thứ ba, các thay đổi được tự động apply bởi controller. Thứ tư, controller liên tục phát hiện drift và reconcile để đưa hệ thống về trạng thái mong muốn.

GitOps: From Principles to Practice



Hình 4. Mô hình GitOps

So với mô hình pipeline trực tiếp deploy vào cluster, GitOps có ưu điểm là giảm quyền truy cập trực tiếp từ CI vào môi trường chạy. CI có thể chỉ cần quyền cập nhật repository cấu hình, còn controller trong cluster chịu trách nhiệm apply. Rollback cũng trở nên tự nhiên hơn vì có thể revert commit trong Git.

Kiến trúc ArgoCD

ArgoCD là một công cụ GitOps cho Kubernetes. Kiến trúc ArgoCD gồm API Server, Repository Server, Application Controller, Redis và UI/CLI. API Server cung cấp giao diện cho UI, CLI và webhook. Repository Server chịu trách nhiệm clone repository và render manifest từ Helm, Kustomize hoặc plain YAML. Application Controller so sánh desired state với live state và thực hiện sync. Redis dùng để cache thông tin. UI và CLI giúp người vận hành quan sát trạng thái application, diff manifest, sync, rollback hoặc xem event.

ArgoCD quản lý ứng dụng thông qua đối tượng Application. Một Application chỉ định source repository, revision, path/chart, destination cluster/namespace và sync policy. Sync policy có thể manual hoặc automated. Automated sync có thể bật prune để xóa resource không còn trong Git và selfHeal để sửa drift do thay đổi thủ công trong cluster.

2.5 AWS EKS

Managed Kubernetes là mô hình trong đó cloud provider vận hành một phần hoặc toàn bộ control plane Kubernetes. AWS Elastic Kubernetes Service là dịch vụ managed Kubernetes của Amazon Web Services. EKS giúp người dùng chạy Kubernetes trên AWS mà không cần tự vận hành API Server, etcd và nhiều phần control plane phức tạp.

Người dùng vẫn quản lý workload, node group, networking, IAM, storage, ingress và các addon theo nhu cầu.

Kiến trúc EKS thường gồm EKS control plane do AWS quản lý, worker node chạy trên EC2 hoặc Fargate, VPC network, IAM integration, security group, load balancer, EBS/EFS storage và CloudWatch logging. EKS tích hợp với AWS Load Balancer Controller để tạo Application Load Balancer hoặc Network Load Balancer từ Kubernetes resource. EKS cũng tích hợp với IAM Roles for Service Accounts để cấp quyền AWS theo từng service account thay vì dùng credential tĩnh.

Một cụm EKS tốt cần chú ý đến network, IAM, logging và security. Worker node thường đặt trong private subnet. Public load balancer nằm ở public subnet. Database hoặc tài nguyên stateful nên được bảo vệ bằng security group và private network. Control plane logging giúp audit và debug. Secret encryption bằng KMS giúp bảo vệ secret lưu trong Kubernetes.

2.6 Các công nghệ DevOps sử dụng

2.6.1 GitHub Actions



GitHub Actions

Hình 5. Github Actions

GitHub Actions là nền tảng CI/CD được tích hợp trực tiếp với GitHub. Trong dự án, GitHub Actions được dùng để tự động hóa toàn bộ quá trình từ kiểm thử đến GitOps promotion. Các job chạy song song khi có thể để rút ngắn thời gian pipeline. Backend test chạy ESLint và Jest; frontend test chạy Vitest; Trivy filesystem scan kiểm tra dependency và misconfiguration; Gitleaks quét secret trong lịch sử Git; SonarCloud phân tích chất lượng mã nguồn; Docker Buildx build image backend/frontend; Trivy image scan kiểm tra image; cuối cùng job update config repo cập nhật image tag.

2.6.2 Terraform



Hình 6. HashiCorp Terraform

Terraform được dùng để mô tả hạ tầng AWS bằng mã nguồn. Dự án chia Terraform thành nhiều thư mục `envs`, mỗi thư mục đại diện cho một state. Cách tổ chức này giúp dễ hiểu thứ tự phụ thuộc và giảm rủi ro khi thay đổi từng phần. Ví dụ, state network tạo VPC và subnet; state EKS đọc output network để tạo cluster; state RDS đọc output network và EKS để cấu hình security group; state DNS dùng ACM và Route53 để cấu hình domain.

2.6.3 Helm



Hình 7. Helm

Helm giúp template hóa Kubernetes manifest để dùng chung cấu trúc nhưng thay đổi giá trị theo môi trường. Chart `task-manager` có các template cho backend, frontend, service, ingress, ExternalSecret, migration Job, ServiceMonitor, ResourceQuota, LimitRange và NetworkPolicy. Các file values như `values.yaml`, `values-aws-dev.yaml`, `values-prod.yaml` quyết định bật/tắt component, image tag, domain, secret name và rollout strategy.

2.6.4 Kustomize



Hình 8. Kustomize

Kustomize xuất hiện trong config repo với vai trò quản lý base/overlay manifest cho một số phần ứng dụng hoặc platform. Helm phù hợp cho chart có nhiều biến cấu hình, còn Kustomize phù hợp khi cần patch manifest theo môi trường.

2.6.5 ArgoCD



Hình 9. ArgoCD

ArgoCD là công cụ Continuous Delivery theo GitOps cho Kubernetes. ArgoCD theo dõi repository cấu hình, so sánh desired state với live state và đồng bộ khi có thay đổi. Trong dự án, ArgoCD Application `task-manager-dev` đọc Helm chart từ config repo, dùng `values.yaml` và `values-aws-dev.yaml`, deploy vào namespace `task-manager-dev`, bật automated sync, prune, selfHeal và retry.

Dự án cũng dùng app-of-apps pattern. File `bootstrap/root-app.yaml` là manifest thủ công duy nhất cần apply ban đầu. Root app đọc thư mục `platform/argocd/apps` và tạo các ArgoCD Application khác như `task-manager-dev`, `kube-prometheus-stack`, `argo-rollouts`, `monitoring-extras` và `grafana-dashboards`.

2.6.6 Argo Rollouts

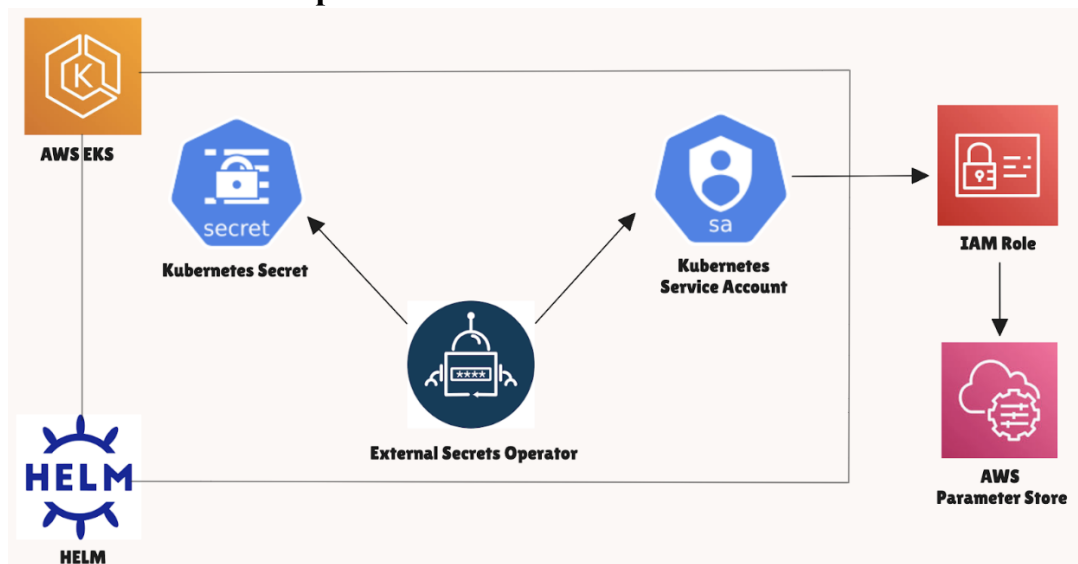


Hình 10. Argo Rollouts

Argo Rollouts là controller mở rộng Kubernetes để hỗ trợ blue/green, canary, analysis và rollback nâng cao. Thay vì Deployment thường, người dùng khai báo Rollout. Rollout có thể tham chiếu Service stable, canary, active hoặc preview. AnalysisTemplate cho phép kiểm tra metrics từ Prometheus, web endpoint hoặc các provider khác để quyết định rollout có tiếp tục hay không.

Các thành phần chính của Argo Rollouts gồm Rollout controller, Rollout resource, AnalysisTemplate, AnalysisRun và các Service dùng để route traffic. Với blue/green, Rollout quản lý active service và preview service. Với canary, Rollout có thể phối hợp với ingress controller hoặc service mesh để chia traffic theo trọng số.

2.6.7 External Secrets Operator



Hình 11. External Secrets Operator

External Secrets Operator là một Kubernetes operator đồng bộ dữ liệu từ external secret store vào Kubernetes Secret. Người vận hành khai báo SecretStore hoặc ClusterSecretStore để mô tả backend secret store, sau đó khai báo ExternalSecret để

chỉ định secret nào cần lấy, key/property nào cần đọc và Kubernetes Secret nào cần tạo. Operator sẽ reconcile định kỳ để đồng bộ dữ liệu.

Kiến trúc này giúp tách nơi lưu secret thật khỏi manifest Kubernetes. Git chỉ lưu reference đến secret, không lưu giá trị nhạy cảm. Pod vẫn dùng Kubernetes Secret theo cách quen thuộc thông qua environment variable hoặc volume mount. Đồng thời, external secret store có thể cung cấp audit log, rotation và policy chi tiết hơn.

2.6.8 Prometheus và Grafana



Hình 12. Prometheus & Grafana

Prometheus là hệ thống metrics mã nguồn mở. Nó hoạt động chủ yếu theo mô hình pull: Prometheus định kỳ scrape metrics từ các target HTTP. Metrics được lưu theo time series, mỗi series có tên metric và tập label. PromQL là ngôn ngữ truy vấn của Prometheus, dùng để tính rate, histogram quantile, aggregation và alert condition.

Kiến trúc Prometheus gồm Prometheus server, target exporters, service discovery, TSDB, rule engine và Alertmanager. Rule engine có thể tạo recording rules để tính sẵn query phức tạp hoặc alerting rules để phát cảnh báo. Alertmanager nhận alert từ Prometheus, gom nhóm, deduplicate và gửi đến email, Slack, webhook hoặc hệ thống quản lý sự cố.

Grafana là nền tảng hiển thị dashboard và phân tích dữ liệu. Grafana không chỉ dùng với Prometheus mà còn hỗ trợ nhiều datasource như Loki, Elasticsearch, InfluxDB, CloudWatch và PostgreSQL. Với Prometheus, Grafana gửi PromQL query đến Prometheus và hiển thị kết quả dưới dạng graph, stat, table, heatmap hoặc gauge

Một phương pháp phổ biến để quan sát HTTP service là RED method: Rate, Errors và Duration. Rate đo số request mỗi giây. Errors đo tỷ lệ request lỗi. Duration đo thời gian xử lý request. Với hệ thống hạ tầng, USE method cũng thường được dùng: Utilization, Saturation và Errors.

2.6.9 Trivy



Hình 13. Trivy

Trivy là công cụ quét bảo mật mã nguồn mở, thường được dùng để phát hiện lỗ hổng trong dependency, container image, filesystem, Infrastructure as Code và Kubernetes manifest. Trivy hỗ trợ nhiều kiểu quét như vulnerability scanning, misconfiguration scanning, secret scanning, license scanning và SBOM.

Về cơ chế hoạt động, Trivy phân tích target cần quét, nhận diện hệ điều hành, package manager, thư viện ứng dụng hoặc file cấu hình, sau đó đối chiếu với cơ sở dữ liệu lỗ hổng để phát hiện CVE. Kết quả thường gồm tên lỗ hổng, package bị ảnh hưởng, phiên bản hiện tại, mức độ nghiêm trọng và phiên bản khuyến nghị để khắc phục.

Kiến trúc xử lý của Trivy có thể hiểu theo ba bước: thu thập và phân tích target, so khớp với vulnerability database, sau đó xuất báo cáo. Trivy hỗ trợ nhiều định dạng output như table, JSON, template và SARIF. Nhờ đó, công cụ này dễ tích hợp vào pipeline CI/CD hoặc các hệ thống quản lý bảo mật.

2.6.10 SonarQube



Hình 14. SonarQube

SonarQube là nền tảng phân tích chất lượng và bảo mật mã nguồn tĩnh. Công cụ này kiểm tra mã nguồn mà không cần chạy ứng dụng, nhằm phát hiện bug, vulnerability, security hotspot, code smell, duplication và mức độ test coverage. SonarQube thường

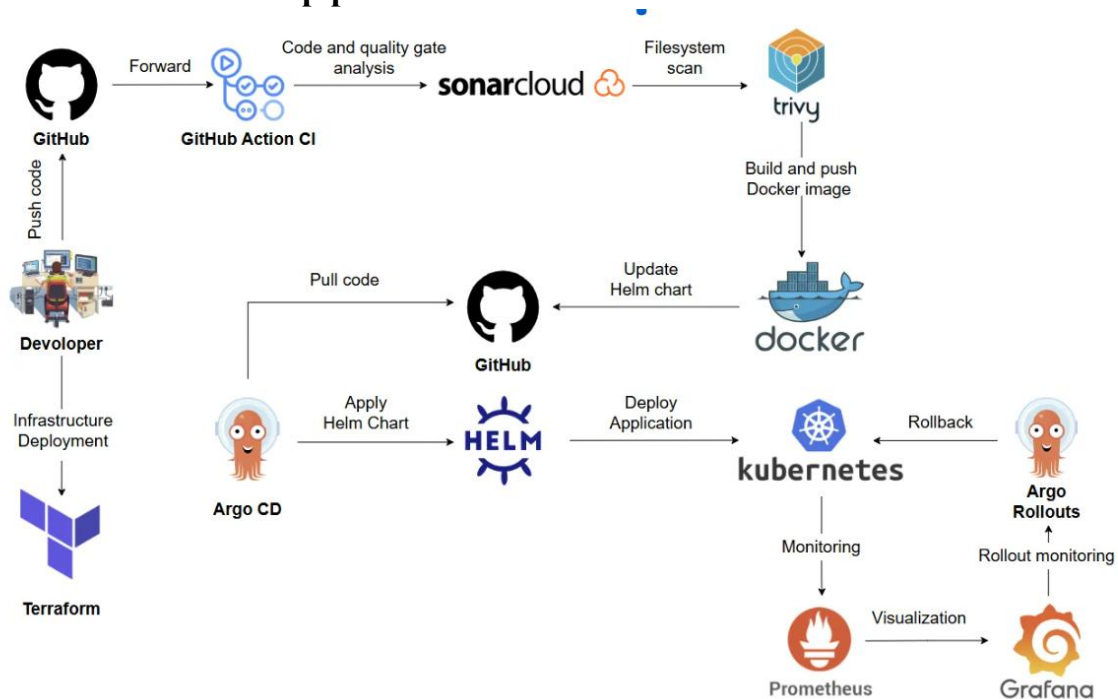
được dùng để nâng cao chất lượng mã nguồn và giảm rủi ro bảo mật ngay từ giai đoạn phát triển.

Kiến trúc SonarQube gồm SonarQube Server, Scanner, Database và giao diện Web. Scanner phân tích mã nguồn và gửi kết quả lên server. Server xử lý kết quả, áp dụng rule, tính toán metric và lưu dữ liệu vào database. Giao diện Web cho phép người dùng xem issue, quality gate, coverage và lịch sử chất lượng của dự án.

SonarQube sử dụng Quality Profile để xác định bộ rule áp dụng cho từng ngôn ngữ lập trình. Ngoài ra, Quality Gate được dùng để đánh giá mã nguồn có đạt tiêu chuẩn chất lượng hay không dựa trên các chỉ số như bug, vulnerability, duplication và coverage. Nhờ đó, SonarQube đóng vai trò như một lớp kiểm soát chất lượng tự động trong quy trình phát triển phần mềm.

Chương 3. TRIỂN KHAI

3.1 Kiến trúc CI/CD pipeline



Hình 15. Kiến trúc CI/CD pipeline

3.1.1 Mục tiêu của CI/CD pipeline

CI/CD pipeline được xây dựng để tự động hóa quá trình từ lúc lập trình viên đưa mã nguồn lên Git cho đến khi image mới được promote sang môi trường triển khai. Pipeline không chỉ build ứng dụng, mà còn đóng vai trò như một cổng kiểm soát chất lượng trước khi artifact được phát hành.

Các mục tiêu chính của pipeline gồm:

- Tự động chạy kiểm thử cho backend và frontend sau mỗi thay đổi mã nguồn.
- Phát hiện lỗi coding convention, lỗi test và lỗi chất lượng mã nguồn trước khi build image.
- Quét dependency, mã nguồn, secret và container image để phát hiện rủi ro bảo mật.
- Build Docker image cho backend và frontend theo cùng một chuẩn.
- Push image lên Docker Hub chỉ khi thay đổi đã được merge/push vào branch triển khai.
- Cập nhật GitOps repository bằng image tag mới thay vì deploy trực tiếp vào Kubernetes cluster.

3.1.2 Trigger và phạm vi chạy pipeline

Workflow chính nằm tại [NT548-DevOps/.github/workflows/ci.yml](#).

```

on:
  push:
    branches: [main, develop, vantai]
    paths-ignore:
      - '**.md'
      - 'docs/**'
  pull_request:
    branches: [main]
  workflow_dispatch:

```

Hình 16. Các trường hợp trigger CI workflow

Pipeline được kích hoạt trong ba trường hợp:

- Push lên các branch main, develop, vantai.
- Pull request vào branch main.
- Chạy thủ công thông qua workflow_dispatch.

Workflow bỏ qua các thay đổi chỉ liên quan tới tài liệu như `\*\*.md` và `docs/\*\*`. Điều này giúp tiết kiệm tài nguyên CI vì những thay đổi tài liệu không cần build lại ứng dụng hoặc push image.

Pipeline cũng dùng cấu hình concurrency:

```

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

```

Hình 17. Cấu hình concurrency cho workflow

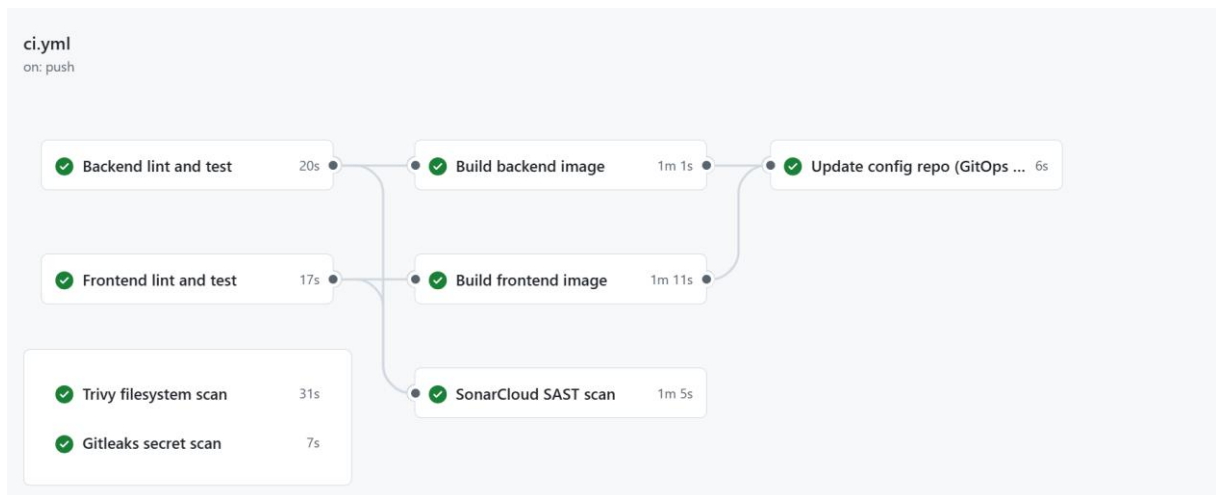
Nhờ đó, nếu một branch có nhiều commit được push liên tục, các workflow cũ trên cùng branch sẽ bị hủy. Pipeline chỉ giữ lại lần chạy mới nhất, tránh tình trạng build artifact từ commit đã lỗi thời.

3.1.3 Kiến trúc các stage trong pipeline

Pipeline được chia thành các stage theo quan hệ phụ thuộc, trong đó stage 2 và stage 3 chạy song song, còn lại thì chạy theo thứ tự nhỏ đến lớn

Stage	Job chính	Vai trò
Stage 1	backend-test, frontend-test, sca-scan, secret-scan	Kiểm thử ứng dụng và thu tín hiệu bảo mật ban đầu
Stage 2	sast-scan	Phân tích chất lượng mã nguồn bằng SonarCloud, dùng coverage từ test job
Stage 3	build-backend, build-frontend	Build Docker image local, scan image, sau đó push image nếu là branch `main`
Stage 4	update-config-repo	Promote image bằng cách cập nhật tag trong GitOps repo

Bảng 1. Danh sách các Stage trong CI pipeline

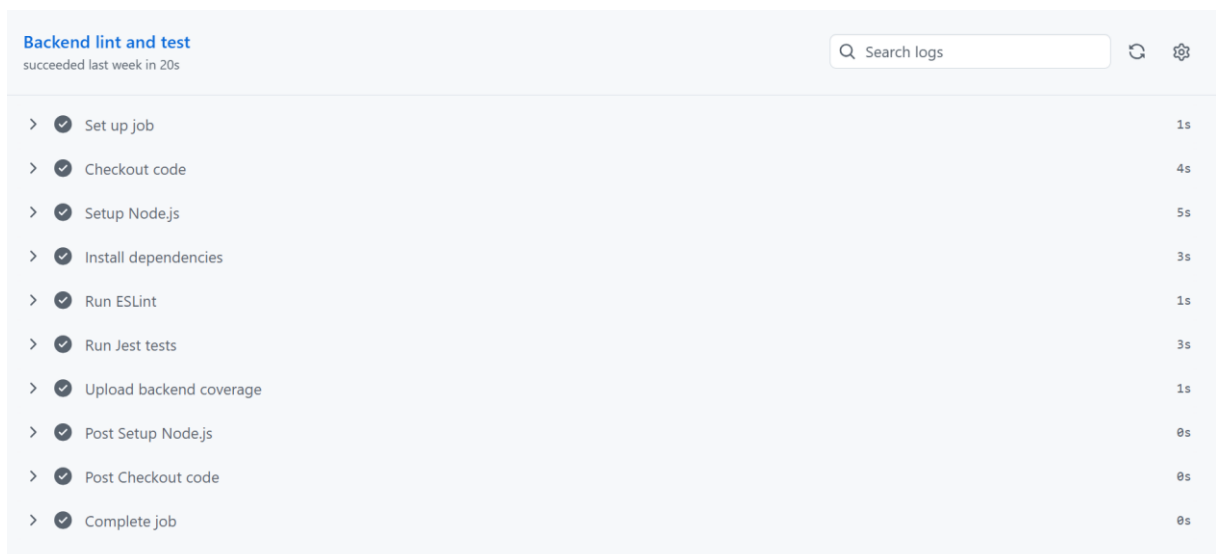


Hình 18. Các stage trong CI pipeline

Build backend image phụ thuộc vào backend test, build frontend phụ thuộc vào frontend test. SonarCloud dùng kết quả Coverage của cả backend và frontend test để chạy sast scan. Cách chia này giúp pipeline nhanh hơn và dễ xác định lỗi thuộc phía nào.

3.1.4 Kiểm thử và kiểm soát chất lượng mã nguồn

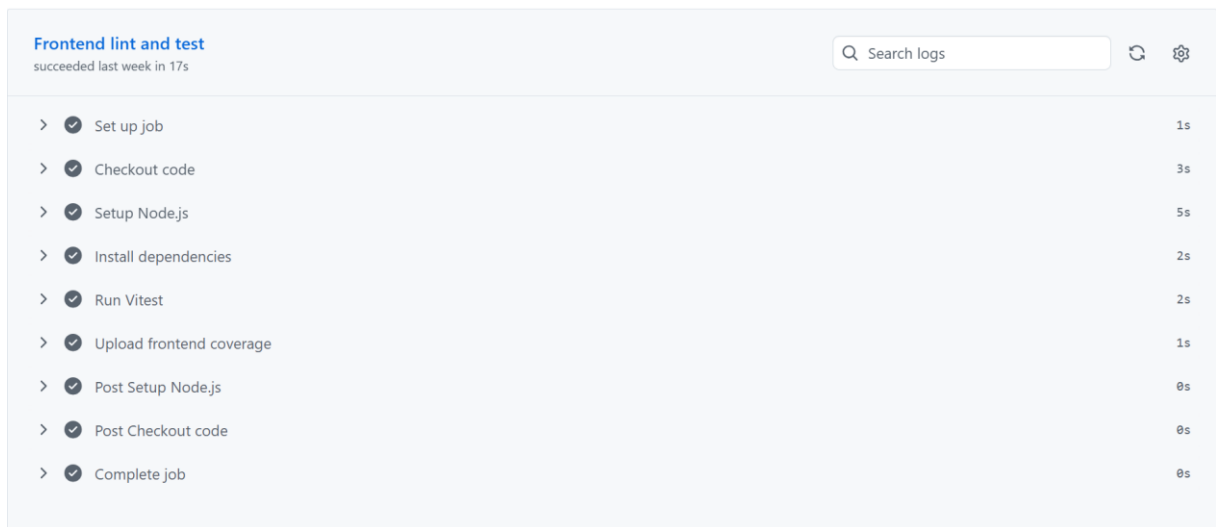
Backend test chạy trong thư mục ``app/mono/backend``. Job thực hiện các bước: checkout code, setup Node.js, cache npm, chạy ``npm ci``, chạy ESLint và chạy Jest test.



Hình 19. Backend lint and test

Các biến môi trường dùng cho test được cấu hình riêng, ví dụ ``DATABASE_URL`` là giá trị giả để test không phụ thuộc vào database thật.

Frontend test chạy trong thư mục ``app/mono/frontend``. Job thực hiện setup Node.js, cache npm, ``npm ci`` và chạy Vitest.



Hình 20. Frontend lint and test

Cả backend và frontend đều upload coverage artifact để SonarCloud dùng ở stage sau.

SonarCloud scan chạy sau khi hai test job hoàn thành. Cấu hình trong sonar-project.properties xác định rõ thư mục source, thư mục test, pattern test file, lcov report path và exclusion. Nhờ vậy phần phân tích chỉ tập trung vào mã nguồn cần đánh giá, không quét node_modules, dist, build hoặc coverage output.

```
NT548-DevOps > sonar-project.properties
1  # sonar-project.properties
2  # SonarCloud configuration cho NT548 Task Manager
3
4  ///
5  sonar.projectKey=nt548-devops
6  sonar.organization=devop-projects-task-manager
7
8  # — Project metadata —
9  sonar.projectName=NT548 Task Manager
10 sonar.projectVersion=1.0
11
12 # — Source code paths —
13 sonar.sources=app/mono/backend/src,app/mono/frontend/src
14 sonar.tests=app/mono/backend/tests,app/mono/frontend/src
15 sonar.test.inclusions=/**/*.test.js,/**/*.test.jsx
16
17 # — Coverage reports (Jest và Vitest đã sinh sẵn lcov.info) —
18 sonar.javascript.lcov.reportPaths=app/mono/backend/coverage/lcov.info,app/mono/frontend/coverage/lcov.info
19
20 # — Exclusions (không scan các file này) —
21 sonar.exclusions=**/node_modules/**,**/*.test.js,**/*.test.jsx,**/coverage/**,**/dist/**,**/build/**
22
23 # — Encoding —
24 sonar.sourceEncoding=UTF-8
```

Hình 21. Cấu hình sonar-project.properties cho SonarQube

3.1.5 Các lớp scan bảo mật trong pipeline

Pipeline có nhiều lớp scan để tăng độ bao phủ:

1. Chạy lint và jest test kiểm tra backend/frontend

```

backend-test:
  name: Backend lint and test
  runs-on: ubuntu-latest
  timeout-minutes: 10

  defaults:
    run:
      working-directory: ./app/mono/backend

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'
        cache-dependency-path: app/mono/backend/package-lock.json

    - name: Install dependencies
      run: npm ci

    - name: Run ESLint
      run: npm run lint

    - name: Run Jest tests
      run: npm test
      env:
        NODE_ENV: test
        JWT_SECRET: ci-test-secret-not-used-in-prod
        JWT_EXPIRES_IN: 1h
        DATABASE_URL: postgres://fake:fake@localhost:5432/fake
        PORT: 3000

    - name: Upload backend coverage
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: backend-coverage
        path: app/mono/backend/coverage/
        retention-days: 7

```

Hình 22. Source code quy trình kiểm thử backend/frontend

Quy trình kiểm thử backend/fronten:

- Checkout code: runner tiến hành kéo toàn bộ mã nguồn của repository để các bước sau có thể cài dependency, lint và chạy test.
- Setup Node.js: dùng action chính thức của GitHub để cài Node.js cho môi trường CI.

- node-version: `${{ env.NODE_VERSION }}` nghĩa là phiên bản Node.js được lấy từ biến môi trường chung của workflow, giúp toàn pipeline dùng cùng một version.
 - cache: 'npm' bật cache cho npm để các lần chạy sau cài dependency nhanh hơn.
 - cache-dependency-path: `app/mono/backend/package-lock.json` nghĩa là cache npm sẽ dựa trên file lock của backend. Khi `package-lock.json` thay đổi thì cache được cập nhật lại.
 - Install dependencies: chạy `npm ci` để cài dependency backend.
 - `npm ci` phù hợp với CI hơn `npm install` vì nó cài đúng theo `package-lock.json`.
 - Nếu dependency trong `package.json` và `package-lock.json` không khớp, bước này sẽ fail, giúp phát hiện lỗi dependency sớm.
 - Run ESLint: chạy `npm run lint` để kiểm tra coding convention và lỗi static code trong source backend.
 - Nếu có lỗi lint nghiêm trọng, job sẽ fail trước khi chạy test.
 - Bước này giúp đảm bảo mã nguồn backend có format và chất lượng tối thiểu trước khi build image.
 - Run Jest tests: chạy `npm test` để thực thi test backend bằng Jest.
 - `NODE_ENV`: test cho biết ứng dụng đang chạy trong môi trường kiểm thử.
 - `JWT_SECRET`: `ci-test-secret-not-used-in-prod` là secret giả chỉ dùng trong CI, không phải secret production.
 - `JWT_EXPIRES_IN`: 1h cấu hình thời gian hết hạn JWT cho test.
 - `DATABASE_URL`: `postgres://fake:fake@localhost:5432/fake` là database URL giả để test có đủ biến môi trường cần thiết.
 - `PORT`: 3000 cấu hình port cho backend khi test cần khởi tạo app.
 - Các biến này giúp backend khởi tạo được trong môi trường CI mà không cần dùng secret thật hoặc database thật, phù hợp cho unit test/mock test.
 - Upload backend coverage: upload thư mục coverage của backend thành artifact để các job sau có thể sử dụng.
 - `if`: `always()` nghĩa là vẫn upload coverage kể cả khi test fail. Điều này hữu ích vì khi test fail vẫn có thể xem được phần coverage hoặc artifact debug nếu file được sinh ra.
 - `uses`: `actions/upload-artifact@v4` dùng action chính thức để lưu artifact.
 - `name`: `backend-coverage` đặt tên artifact là `backend-coverage`.
 - `path`: `app/mono/backend/coverage/` là thư mục chứa báo cáo coverage do Jest sinh ra.
 - `retention-days`: 7 nghĩa là artifact được giữ trong 7 ngày rồi tự xóa để tránh lưu trữ quá lâu.
2. Trivy filesystem scan quét repository ở mức filesystem, tập trung vào dependency, lockfile và một số misconfiguration.

```

sca-scan:
  name: Trivy filesystem scan
  runs-on: ubuntu-latest
  timeout-minutes: 5

  permissions:
    contents: read
    security-events: write

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Run Trivy filesystem scan
      uses: aquasecurity/trivy-action@v0.36.0
      with:
        scan-type: 'fs'
        scan-ref: '.'
        format: 'sarif'
        output: 'trivy-fs-results.sarif'
        severity: 'CRITICAL,HIGH'
        exit-code: '0'

    - name: Upload Trivy fs SARIF
      if: always()
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: 'trivy-fs-results.sarif'
        category: 'trivy-fs'

```

Hình 23. Source code quy trình scan file system

Quy trình scan file system:

- Checkout code: runner tiến hành kéo toàn bộ mã nguồn của repository để quét ở bước sau
- Run Trivy filesystem scan: Dùng GitHub Action chính thức của Trivy, với các tham số có thể tùy chỉnh:
 - Scan type: fs nghĩa là quét filesystem, tức là quét nội dung repository thay vì quét Docker image.
 - scan-ref: '.' nghĩa là quét toàn bộ thư mục hiện tại.
 - format: 'sarif' nghĩa là kết quả được xuất theo định dạng SARIF
 - output: 'trivy-fs-results.sarif' nghĩa là kết quả sẽ được xuất ra file trivy-fs-results.sarif
 - severity: 'CRITICAL,HIGH' Chỉ lấy các lỗi hỏng mức HIGH và CRITICAL, các mức thấp hơn như MEDIUM, LOW, UNKNOWN sẽ không được đưa vào báo cáo này.
 - Exit code: '0' nghĩa là dù Trivy phát hiện lỗi hỏng HIGH/CRITICAL thì job vẫn không fail. Nói cách khác, job này đang chạy theo kiểu report-only, tức

là chỉ phục vụ trong giai đoạn đồ án, sau này nếu triển khai thật sẽ set là 1 để job thực sự fail nếu quét ra lỗi nghiêm trọng

- Upload Trivy fs SARIF: Dùng action của GitHub để upload file báo cáo vừa tạo lên github code scanning

Job được set quyền tối thiểu để chạy:

- Contents: read chỉ cho phép đọc mã nguồn
 - Secirity-events: write được phép ghi vào để làm báo cáo trong tab security
3. SonarCloud SAST phân tích mã nguồn để phát hiện bug, vulnerability, code smell và theo dõi coverage.

```
sast-scan:
  name: SonarCloud SAST scan
  runs-on: ubuntu-latest
  needs: [backend-test, frontend-test]
  timeout-minutes: 10

  steps:
    - name: Checkout code
      uses: actions/checkout@v4
      with:
        fetch-depth: 0

    - name: Download backend coverage
      uses: actions/download-artifact@v4
      with:
        name: backend-coverage
        path: app/mono/backend/coverage/

    - name: Download frontend coverage
      uses: actions/download-artifact@v4
      with:
        name: frontend-coverage
        path: app/mono/frontend/coverage/

    - name: SonarCloud Scan
      uses: SonarSource/sonarqube-scan-action@v5
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        SONAR_HOST_URL: 'https://sonarcloud.io'
```

Hình 24. Source code quy trình scan SAST SonarQube

Quy trình scan SAST SonarQube:

- Checkout code: runner tiến hành kéo toàn bộ mã nguồn của repository để quét ở bước sau
- Download backend/frontend coverage: Tiến hành tải artifact coverage do backend-test/frontend-test upload trước đó.
- SonarCloud Scan: Tiến hành quét sonarQube bằng SonarCloud scanner. Scanner sẽ đọc cấu hình từ file sonar-project.properties để chạy

4. Gitleaks quét toàn bộ git history để phát hiện secret từng bị commit.

```
secret-scan:
  name: Gitleaks secret scan
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    - name: Install Gitleaks
      run: |
        VERSION=8.18.4
        wget https://github.com/gitleaks/gitleaks/releases/download/v${VERSION}/gitleaks_${VERSION}_linux_x64.tar.gz
        tar -xzf gitleaks_${VERSION}_linux_x64.tar.gz
        sudo mv gitleaks /usr/local/bin/

    - name: Run Gitleaks scan
      run: |
        gitleaks detect --source . --verbose --redact
```

Hình 25. Source code scan secrets

Quy trình scan secrets:

- Checkout code: runner tiến hành kéo toàn bộ mã nguồn của repository để quét ở bước sau.
 - Install Gitleaks: tải gitleaks về runner để chạy scan
 - Run Gitleaks scan: Tiến hành chạy lệnh gitleaks detect với các tham số:
 - -- source . : quét toàn bộ mã nguồn
 - -- verbose: In log chi tiết hơn, giúp dễ biết Gitleaks đang quét gì và phát hiện vấn đề ở đâu.
 - -- redact: đây là tham số quan trọng, giúp che bớt các secret trong log, bởi vì log console của CI cũng có thể là một nơi rò rỉ secret.
5. Trivy image scan quét container image sau khi image đã được build local.

```

build-backend:
  name: Build backend image
  needs: backend-test
  runs-on: ubuntu-latest
  timeout-minutes: 15

  permissions:
    contents: read
    packages: write
    security-events: write

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    # — STEP 1: Build LOCAL image (chưa push) để scan —
    - name: Build image (local, for scanning)
      uses: docker/build-push-action@v5
      with:
        context: ./app/mono/backend
        file: ./app/mono/backend/Dockerfile
        push: false
        tags: nt548-backend:scan
        load: true
        cache-from: type=gha
        cache-to: type=gha,mode=max

    # — STEP 2: Scan image với Trivy —
    - name: Trivy image scan
      uses: aquasecurity/trivy-action@v0.36.0
      with:
        image-ref: 'nt548-backend:scan'
        format: 'sarif'
        output: 'trivy-image-backend.sarif'
        severity: 'CRITICAL,HIGH'
        exit-code: '0'

    - name: Upload Trivy image SARIF
      if: always()
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: 'trivy-image-backend.sarif'
        category: 'trivy-image-backend'

    # — STEP 3: Login Docker Hub (chỉ trên main) —
    - name: Login to Docker Hub
      if: github.event_name == 'push' && github.ref == 'refs/heads/main'
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }

    # — STEP 4: Generate multi-tag metadata —
    - name: Extract image metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ secrets.DOCKERHUB_USERNAME }/nt548-backend
        tags: |
          type=raw,value=latest,enable={{is_default_branch}}
          type=sha,prefix=,format=short
          type=ref,event=branch
          type=semver,pattern={{version}}

    # — STEP 5: Push image (chỉ trên main, tận dụng cache) —
    - name: Push image with multiple tags
      if: github.event_name == 'push' && github.ref == 'refs/heads/main'
      uses: docker/build-push-action@v5
      with:
        context: ./app/mono/backend
        file: ./app/mono/backend/Dockerfile
        push: true
        tags: ${ steps.meta.outputs.tags }
        labels: ${ steps.meta.outputs.labels }
        cache-from: type=gha # Reuse cache → build nhanh ~10s

```

Hình 26. Source code quy trình scan Docker image backend local

- Quy trình scan Docker image backend local:
- Build image local để scan: runner build Docker image backend từ thư mục `app/mono/backend` nhưng chưa push lên Docker Hub. Image được gắn tag tạm là `nt548-backend:scan`.
 - `context`: `./app/mono/backend` nghĩa là Docker build context nằm trong thư mục backend.
 - `file`: `./app/mono/backend/Dockerfile` nghĩa là dùng Dockerfile của backend để build image.
 - `push`: `false` nghĩa là image chỉ được build local, chưa đẩy lên registry.
 - `tags`: `nt548-backend:scan` là tag tạm dùng riêng cho bước scan.
 - `load`: `true` nghĩa là image sau khi build sẽ được load vào Docker daemon local của runner, để Trivy có thể quét được image này.
 - `cache-from` và `cache-to` dùng GitHub Actions cache để tăng tốc các lần build sau.
- Trivy image scan: dùng GitHub Action chính thức của Trivy để quét Docker image backend vừa build.
 - `image-ref`: `'nt548-backend:scan'` nghĩa là Trivy sẽ quét image local có tag `nt548-backend:scan`.
 - `format`: `'sarif'` nghĩa là kết quả scan được xuất theo định dạng SARIF.
 - `output`: `'trivy-image-backend.sarif'` nghĩa là kết quả sẽ được ghi vào file `trivy-image-backend.sarif`.
 - `severity`: `'CRITICAL,HIGH'` chỉ lấy các lỗ hổng mức HIGH và CRITICAL, bỏ qua các mức thấp hơn như MEDIUM, LOW, UNKNOWN.
 - `exit-code`: `'0'` nghĩa là dù Trivy phát hiện lỗ hổng HIGH/CRITICAL thì job vẫn không fail. Job này đang chạy theo kiểu `report-only`, phù hợp giai đoạn đồ án. Nếu triển khai thật, có thể đổi thành `exit-code: '1'` để pipeline fail khi image có lỗi nghiêm trọng.
- Upload Trivy image SARIF: dùng action của GitHub để upload file báo cáo `trivy-image-backend.sarif` lên GitHub Code Scanning.
 - `if`: `always()` nghĩa là luôn cố upload báo cáo dù bước scan trước đó thành công hay thất bại.
 - `sarif_file`: `'trivy-image-backend.sarif'` chỉ định file SARIF cần upload.
 - `category`: `'trivy-image-backend'` giúp GitHub phân loại báo cáo này riêng với các báo cáo khác như `trivy-fs` hoặc `trivy-image-frontend`.

3.1.6 Promotion sang GitOps repo

Job `update-config-repo` chỉ chạy khi workflow được kích hoạt bởi push vào branch `main`.


```

update-config-repo:
  name: Update config repo (GitOps promotion)
  runs-on: ubuntu-latest
  needs: [build-backend, build-frontend]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  timeout-minutes: 5

  steps:
    # — Checkout config repo —————
    - name: Checkout config repo
      uses: actions/checkout@v4
      with:
        repository: Devop-Projects/nt548-config
        token: ${ secrets.CONFIG_REPO_TOKEN }
        path: config-repo
        ref: main

    # — Install yq for YAML manipulation —————
    # yq is a YAML-aware sed (jq for YAML)
    - name: Install yq
      run: |
        sudo wget -qO /usr/local/bin/yq \
          https://github.com/mikefarah/yq/releases/download/v4.44.3/yq_linux_amd64
        sudo chmod +x /usr/local/bin/yq
        yq --version

    # — Extract image tag —————
    # github.sha is the full SHA. We want short SHA (7 chars) to match
    # the tag CI uses in build-backend/build-frontend jobs.
    - name: Extract short SHA
      id: sha
      run: echo "short=$(echo ${ github.sha } | cut -c1-7)" >> $GITHUB_OUTPUT

    # — Update values-aws-dev.yaml —————
    - name: Update backend image tag
      working-directory: config-repo
      run: |
        yq eval -i '.backend.image.tag = "${ steps.sha.outputs.short }"' \
          charts/task-manager/values-aws-dev.yaml

    - name: Update frontend image tag
      working-directory: config-repo
      run: |
        yq eval -i '.frontend.image.tag = "${ steps.sha.outputs.short }"' \
          charts/task-manager/values-aws-dev.yaml

    # — Verify changes —————
    - name: Show diff
      working-directory: config-repo
      run: |
        echo "=== Changes in values-aws-dev.yaml ==="
        git diff charts/task-manager/values-aws-dev.yaml || true

    # — Commit & push —————
    - name: Commit and push
      working-directory: config-repo
      run: |
        # Configure git as CI bot
        git config user.email "ci-bot@nt548-devops.local"
        git config user.name "NT548 CI Bot"

        # Check if there are changes (idempotent)
        if git diff --quiet; then
          echo "✓ No changes to values-aws-dev.yaml (image tag already up-to-date)"
          exit 0
        fi

        # Stage + commit + push
        git add charts/task-manager/values-aws-dev.yaml
        git commit -m "ci(dev): promote image to ${ steps.sha.outputs.short }"

        Source commit: https://github.com/${ github.repository }/commit/${ github.sha }
        Triggered by: ${ github.actor }
        Workflow: ${ github.workflow } #${ github.run_number }

```

```

[skip ci]"

git push origin main
echo "✓ Pushed image tag update to config repo"

# --- Notify status ---
- name: Print summary
  run: |
    echo "=====
    echo "  GitOps Promotion Complete"
    echo "=====
    echo "  Image tag:      ${ steps.sha.outputs.short }"
    echo "  Config repo:      Devop-Projects/nt548-config"
    echo "  Updated file:      charts/task-manager/values-aws-dev.yaml"
    echo "  Next:              ArgoCD will sync within 3 minutes"
    echo "                  https://localhost:8081/applications/task-manager-dev"
    echo "=====

```

Hình 27. Source code quy trình update config repo

Quy trình update config repo:

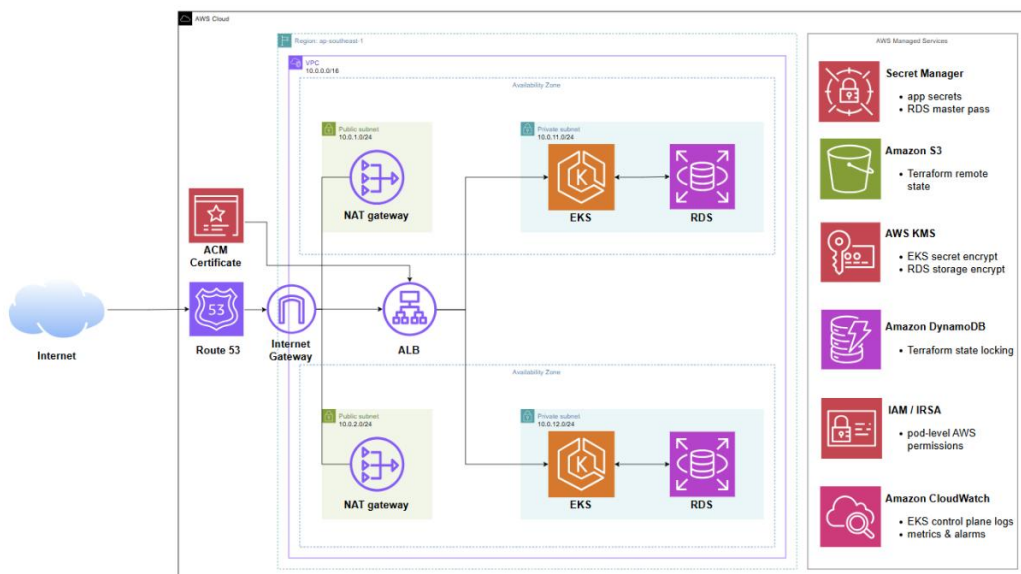
- Điều kiện chạy job:
 - needs: [build-backend, build-frontend] nghĩa là job này chỉ chạy sau khi cả hai job build image backend và frontend hoàn thành.
 - if: github.event_name == 'push' && github.ref == 'refs/heads/main' nghĩa là chỉ chạy khi có push trực tiếp lên nhánh main.
 - Như vậy, pull request hoặc push lên nhánh khác sẽ không tự cập nhật config repo.
 - Đây là cách kiểm soát promotion: chỉ commit đã vào main mới được đưa sang GitOps repo.
- Checkout config repo:
 - Dùng actions/checkout@v4 để clone repository Devop-Projects/nt548-config.
 - token: { secrets.CONFIG_REPO_TOKEN } là token có quyền ghi vào config repo.
 - path: config-repo nghĩa là repo config được checkout vào thư mục config-repo trên runner.
 - ref: main nghĩa là checkout nhánh main của config repo.
 - Điểm quan trọng: job này không sửa repo ứng dụng hiện tại, mà sửa repo GitOps riêng. Đây là đúng tinh thần GitOps.
- Install yq: Runner tải yq là công cụ chỉnh sửa YAML và cấp quyền execute bằng chmod
- Extract short SHA:
 - { github.sha } là full commit SHA của commit đang chạy pipeline.
 - Lệnh cut -c1-7 lấy 7 ký tự đầu để tạo short SHA.
 - Kết quả được ghi vào GITHUB_OUTPUT với key short.
 - Các step sau có thể dùng bằng cú pháp:
 - { steps.sha.outputs.short }

- Mục đích là để image tag trong config repo khớp với tag mà job build image đã push lên Docker Hub.
- Update backend/frontend image tag:
 - Lệnh `yq eval -i` chỉnh trực tiếp file YAML.
 - Trường được cập nhật là `.backend.image.tag`, giá trị mới là short SHA của commit hiện tại.
- Show diff:
 - Chạy `git diff charts/task-manager/values-aws-dev.yaml` để in ra thay đổi vừa thực hiện.
 - Bước này giúp log CI hiển thị rõ tag cũ và tag mới.
 - `|| true` giúp step không fail nếu không có diff hoặc command trả trạng thái không mong muốn.
- Commit and push:
 - Trước tiên job cấu hình Git identity cho CI bot, Sau đó kiểm tra có thay đổi thật không:
 - Nếu image tag đã đúng rồi thì job thoát thành công, không tạo commit rỗng.
 - Nếu có thay đổi, job add file `values-aws-dev.yaml`, commit và push lên main của config repo.
- Print summary:
 - In ra thông tin tổng kết sau khi promotion xong.
 - Gồm image tag, config repo, file được cập nhật và ghi chú rằng ArgoCD sẽ sync sau đó.
 - Link `https://localhost:8081/...` chỉ dùng được nếu người vận hành đang port-forward ArgoCD UI về máy local.

Đây là điểm nối giữa CI và ArgoCD.

Bằng cách commit và push thay đổi này về config repo. ArgoCD theo dõi config repo nên sẽ phát hiện commit mới và đồng bộ phiên bản ứng dụng mới lên cluster.

3.2 Kiến trúc hệ thống AWS



Hình 28. Kiến trúc hệ thống AWS

Hệ thống được triển khai tại region `ap-southeast-1`. Kiến trúc AWS được thiết kế theo mô hình có lớp mạng riêng, lớp Kubernetes để chạy workload, lớp database managed service, lớp DNS/TLS và lớp quản lý secret. Các thành phần chính gồm VPC, public/private subnet, NAT Gateway, EKS, RDS PostgreSQL, AWS Secrets Manager, KMS, Route53, ACM và Application Load Balancer.

Luồng truy cập tổng quát:

- Người dùng truy cập domain của ứng dụng thông qua HTTPS.
- Route53 trả domain về Application Load Balancer.
- ALB nhận request và chuyển traffic vào Kubernetes Ingress.
- Ingress định tuyến `/api` tới backend service và `/` tới frontend service.
- Backend chạy trong EKS, kết nối tới RDS PostgreSQL ở private subnet.
- Backend đọc thông tin nhạy cảm thông qua Kubernetes Secret, các Secret này được đồng bộ từ AWS Secrets Manager.
- Metrics và log được thu thập để phục vụ quan sát hệ thống.

3.2.1 Network layer

Network layer gồm một VPC cho môi trường dev, chia thành public subnet và private subnet trên nhiều Availability Zone. Public subnet dành cho các entry point cần nhận traffic từ Internet, chủ yếu là Application Load Balancer. Private subnet dành cho EKS worker node và RDS, giúp workload và database không có public IP trực tiếp.

NAT Gateway được đặt để tài nguyên trong private subnet vẫn có thể đi ra Internet khi cần, ví dụ pull image, tải dependency hoặc gọi dịch vụ AWS public endpoint. Ở môi trường dev, nhóm dùng single NAT Gateway để giảm chi phí. Nếu chuyển sang production, có thể cân nhắc NAT Gateway theo từng Availability Zone để tăng độ sẵn sàng.

Các subnet được gắn tag để AWS Load Balancer Controller có thể tự động chọn subnet khi tạo load balancer. Public subnet có tag phục vụ public ALB, private subnet có tag phục vụ internal load balancer nếu sau này cần.

3.2.2 Kubernetes compute layer bằng EKS

EKS là lớp compute chính của hệ thống. Cluster EKS quản lý control plane, còn workload chạy trên managed node group trong private subnet. Việc đặt worker node trong private subnet giúp giảm bề mặt tấn công vì người dùng không truy cập trực tiếp vào node.

Các thành phần platform chạy trên EKS gồm:

- AWS Load Balancer Controller để biến Kubernetes Ingress thành Application Load Balancer trên AWS.
- External Secrets Operator để đồng bộ secret từ AWS Secrets Manager về Kubernetes Secret.
- ArgoCD để triển khai theo GitOps.
- Argo Rollouts để hỗ trợ blue-green/canary deployment.
- Monitoring stack gồm Prometheus và Grafana.

EKS cũng bật các add-on cơ bản như CoreDNS, kube-proxy, VPC CNI và EBS CSI driver. VPC CNI cho phép pod tích hợp trực tiếp với network của AWS, còn EBS CSI driver phục vụ các workload cần PersistentVolume.

3.2.3 Database layer bằng RDS PostgreSQL

Database của hệ thống dùng Amazon RDS PostgreSQL 15. RDS được đặt trong private subnet và không public ra Internet. Security group của RDS chỉ cho phép inbound port 5432 từ security group của EKS worker node. Cách này phù hợp với Kubernetes vì worker node có thể thay đổi IP khi scale, nhưng security group reference vẫn giữ được quyền truy cập ổn định.

RDS bật encryption at rest bằng KMS, export log PostgreSQL lên CloudWatch và dùng AWS-managed master password. Password được sinh và lưu trong AWS Secrets Manager thay vì hardcode trong Terraform hoặc Helm values.

Với môi trường dev, RDS được cấu hình theo hướng tiết kiệm chi phí: Multi-AZ tắt, deletion protection tắt và skip final snapshot khi destroy. Đây là lựa chọn phù hợp cho demo/lab, nhưng khi triển khai production cần thay đổi theo hướng an toàn hơn.

3.2.4 Secret và encryption layer

Secret không được lưu trực tiếp trong Git dưới dạng giá trị thật. AWS Secrets Manager giữ secret gốc như database credential và JWT secret. Kubernetes chỉ có bản Secret được đồng bộ bởi External Secrets Operator.

KMS được dùng cho các nhu cầu mã hóa at rest, gồm EKS secret encryption, RDS storage encryption và decrypt secret thông qua Secrets Manager. External Secrets Operator được cấp quyền bằng IRSA, nghĩa là service account cụ thể trong Kubernetes được map tới IAM role cụ thể trên AWS. Cách này tránh cấp quyền rộng cho toàn bộ node.

3.2.5 DNS, TLS và endpoint

Route53 quản lý DNS record cho domain ứng dụng và Grafana. ACM cấp TLS certificate bằng DNS validation. Application Load Balancer là endpoint public của hệ thống, nhận traffic HTTP/HTTPS và chuyển request vào workload trong EKS.

Ingress của ứng dụng dùng class `alb`, gắn certificate ARN, bật SSL redirect và khai báo health check path. AWS Load Balancer Controller đọc Ingress này để tạo ALB, target group, listener và rule tương ứng. Sau khi ALB được tạo, Route53 alias record trỏ domain về ALB.

Nhờ kiến trúc này, người dùng chỉ truy cập qua domain HTTPS. Backend, frontend service, node và database đều không cần public trực tiếp.

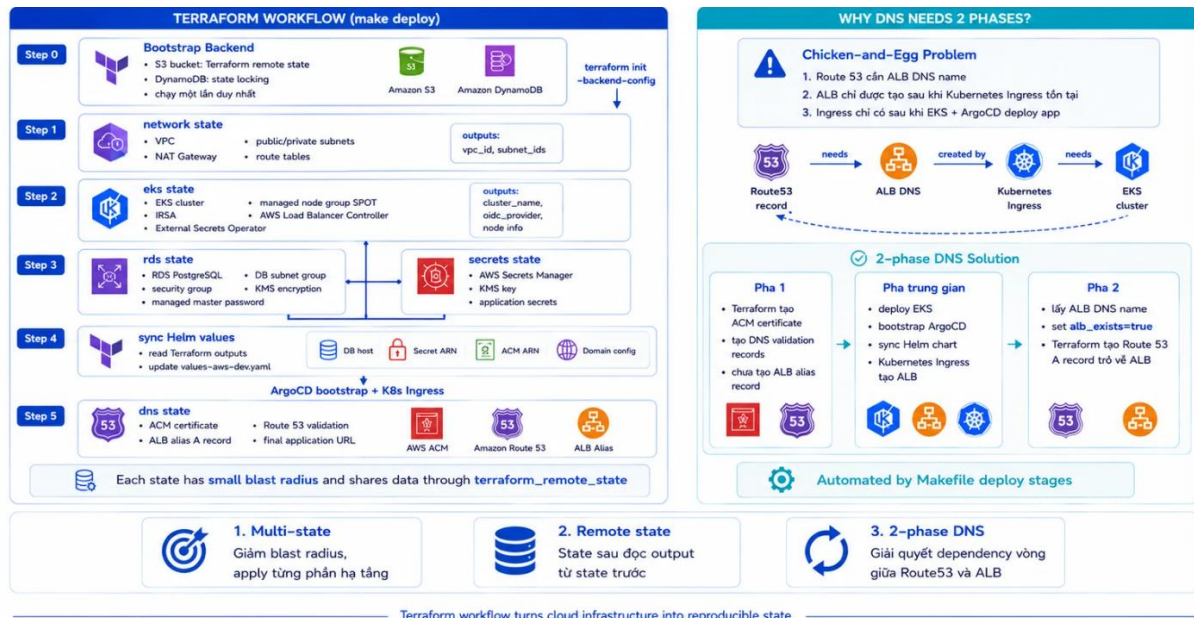
3.2.6 Observability layer

Hệ thống có lớp quan sát gồm CloudWatch, Prometheus và Grafana. CloudWatch nhận log từ EKS control plane và RDS. Prometheus scrape metrics từ backend thông qua ServiceMonitor. Grafana hiển thị dashboard phục vụ quan sát request rate, error rate, latency, trạng thái node và trạng thái backend.

Lớp observability giúp kiểm tra sau deploy, debug lỗi runtime và làm dữ liệu đầu vào cho progressive delivery nếu dùng AnalysisTemplate của Argo Rollouts.

3.3 Xây dựng hạ tầng AWS bằng Terraform

Hạ tầng AWS được triển khai bằng Terraform theo nhiều state. Cách này giúp các phần hạ tầng độc lập tương đối, dễ apply từng phần, dễ destroy khi cần tiết kiệm chi phí và dễ phân tích phụ thuộc.



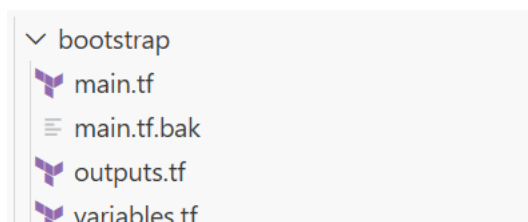
Hình 29. Mô hình Terraform workflow

Các state chính gồm:

- bootstrap: tạo S3 bucket và DynamoDB table dùng cho Terraform remote state.
- network: tạo VPC, subnet, NAT Gateway và các tag mạng cần thiết.
- eks: tạo EKS cluster, node group, KMS key, EKS addons và các controller cấp cluster.
- rds: tạo PostgreSQL RDS, subnet group, security group, parameter group và KMS key.
- secrets: tạo secret ứng dụng trong AWS Secrets Manager.
- dns: tạo ACM certificate, DNS validation record và Route53 record trỏ về ALB.

3.3.1 Terraform remote state

Cấu trúc bootstrap remote state:



Hình 30. Cấu trúc bootstrap remote state

Terraform state được bootstrap bằng S3 bucket và DynamoDB lock table. S3 lưu tfstate, còn DynamoDB lock giúp tránh hai tiến trình Terraform cùng ghi state.

```
bucket      = "thesis-tfstate-113008551898"
region      = "ap-southeast-1"
dynamodb_table = "thesis-tfstate-locks"
encrypt     = true
```

Hình 31. Terraform S3 Bucket tfstate và dynamoDB lock state

Đây là một yêu cầu quan trọng khi hạ tầng có nhiều state và có thể được nhiều thành viên thao tác. State backend được cấu hình trong backend-config.hcl và mỗi state dùng key riêng như dev/network/terraform.tfstate, dev/eks/terraform.tfstate.

```
data "aws_caller_identity" "current" {}

locals {
  account_id = data.aws_caller_identity.current.account_id
  bucket_name = "thesis-tfstate-${local.account_id}"
  table_name  = "thesis-tfstate-locks"
}
```

Hình 32. Dynamic Tên bucket và lock table

Tên bucket và lock table được tạo từ account AWS hiện tại

```
terraform {
  required_version = ">= 1.5"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "local" {
    path = "terraform.tfstate"
  }
}
```

Hình 33. Local backend cho bootstrap state

Bootstrap state dùng local backend để lưu state vì lúc này chưa có S3 bucket, một lí do nữa là nếu S3 hỏng, vẫn có local state để rebuild.

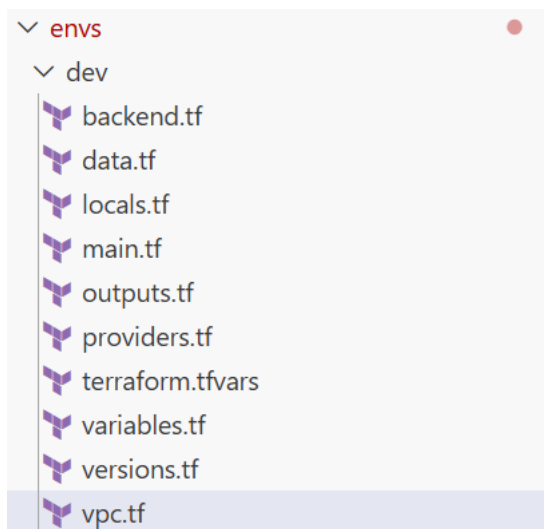
```
NT548-DevOps > infrastructure > envs > rds > backend.tf
1 terraform {
2   | backend "s3" {}
3 }
```

Hình 34. Các state còn lại dùng S3 Bucket lưu tfstate

Sau khi S3 bucket tồn tại, các state khác dùng backend-config.hcl để trỏ vào bucket đó để lưu tfstate.

3.3.2 VPC và network

Cấu trúc network state nằm trong infrastructure/envs/dev:



Hình 35. Cấu trúc network state nằm trong infrastructure/envs/dev

Hệ thống dùng public subnet cho load balancer và private subnet cho worker node hoặc database. NAT Gateway cho phép workload trong private subnet truy cập Internet khi cần pull image hoặc gọi dịch vụ bên ngoài. DNS hostname và DNS support được bật vì đây là yêu cầu quan trọng đối với EKS.

```
# — NAT Gateway —————
enable_nat_gateway = true
single_nat_gateway = var.single_nat_gateway

# — DNS settings —————
enable_dns_hostnames = true
enable_dns_support   = true

public_subnet_tags = {
  "kubernetes.io/role/elb" = 1
  Tier                     = "public"
}

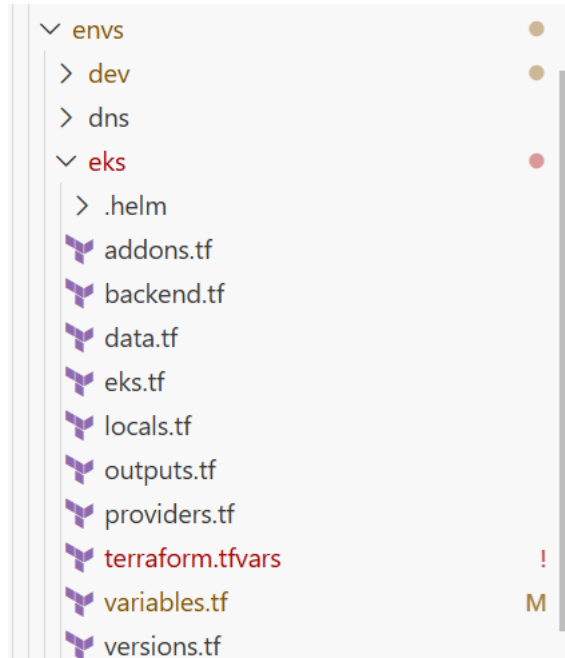
private_subnet_tags = {
  "kubernetes.io/role/internal-elb" = 1
  Tier                             = "private"
}
```

Hình 36. NAT Gateway, DNS settings, public subnet, private subnet

Subnet được gắn tag để AWS Load Balancer Controller tự nhận diện. Public subnet có tag “kubernetes.io/role/elb”, private subnet có tag “kubernetes.io/role/internal-elb”. Nếu thiếu tag này, controller có thể không tạo được load balancer từ Ingress.

3.3.3 EKS cluster

EKS state nằm trong `infrastructure/envs/eks`:



Hình 37. EKS state nằm trong `infrastructure/envs/eks`

EKS cluster được cấu hình với managed node group chạy trong private subnets.

```
eks_managed_node_groups = {
  default = {
    # Naming
    name = "${local.cluster_name}-default"

    # Capacity
    min_size      = var.node_min_size
    max_size      = var.node_max_size
    desired_size  = var.node_desired_size

    # Instance config
    instance_types = var.node_instance_types
    capacity_type  = var.node_capacity_type # SPOT for dev

    # Disk
    disk_size = 30 # GB EBS gp3

    # Labels (cho scheduler)
    labels = {
      role          = "general"
      environment = var.environment
    }

    # Taints (optional - limit pod được schedule)
    # taints = {
    #   spot = {
    #     key   = "spot"
    #     value = "true"
    #     effect = "NO_SCHEDULE"
    #   }
    # }

    tags = local.common_tags
  }
}
```

Hình 38. cấu hình managed node group cho EKS cluster

Cluster endpoint có public access để phục vụ kubectl từ máy cá nhân, đồng thời public CIDR nên được giới hạn theo IP của người triển khai.

```
cluster_endpoint_public_access      = true
cluster_endpoint_public_access_cidrs = var.cluster_endpoint_public_access_cidrs
cluster_endpoint_private_access     = true
```

Hình 39. API Endpoint để kubectl từ máy cá nhân

Cluster có private access từ trong VPC. Addons gồm CoreDNS, kube-proxy, VPC CNI và EBS CSI Driver. VPC CNI bật prefix delegation để tăng khả năng cấp IP cho Pod.

```

cluster_addons = {
  coredns = {
    most_recent = true
  }
  kube-proxy = {
    most_recent = true
  }
  vpc-cni = {
    most_recent = true
    # Tăng số IP per ENI (default 1 IP/pod, max 740 pod/node)
    configuration_values = jsonencode({
      env = {
        ENABLE_PREFIX_DELEGATION = "true"
        WARM_PREFIX_TARGET       = "1"
      }
    })
  }
  aws-ebs-csi-driver = {
    most_recent            = true
    service_account_role_arn = module.ebs_csi_irsa.iam_role_arn
  }
}

```

Hình 40. Cluster addons

Cluster secret encryption được bật bằng KMS key.

```

resource "aws_kms_key" "eks" {
  description          = "EKS secret encryption - ${local.cluster_name}"
  deletion_window_in_days = 7
  enable_key_rotation  = true

  tags = merge(local.common_tags, {
    Name = "${local.cluster_name}-encryption"
  })
}

```

Hình 41. Cluster secret encryption

Control plane logs gồm API, audit, authenticator, controllerManager và scheduler được gửi về CloudWatch với retention 7 ngày cho môi trường dev. Managed node group dùng Spot instance như `t3.medium` để tiết kiệm chi phí.

Ngoài cluster, EKS state còn tạo các addon cập cluster bằng Helm. Ví dụ AWS Load Balancer Controller:

```

resource "helm_release" "aws_load_balancer_controller" {
  name           = "aws-load-balancer-controller"
  repository      = "https://aws.github.io/eks-charts"
  chart          = "aws-load-balancer-controller"
  namespace      = "kube-system"
  version        = "1.8.1" # Check latest at https://github.com/aws/eks-charts

  depends_on = [module.eks]

  wait           = true
  wait_for_jobs  = true
  timeout        = 300 # 5 phút

  set {
    name = "clusterName"
    value = module.eks.cluster_name
  }
  set {
    name = "serviceAccount.create"
    value = "true"
  }
  set {
    name = "serviceAccount.name"
    value = "aws-load-balancer-controller"
  }
  set {
    name = "serviceAccount.annotations.eks\\.amazonaws\\.com/role-arn"
    value = module.aws_load_balancer_controller_irsas.iam_role_arn
  }
  set {
    name = "region"
    value = var.region
  }
  set {
    name = "vpcId"
    value = data.terraform_remote_state.network.outputs.vpc_id
  }
  set {
    name = "replicaCount"
    value = "2"
  }
}

```

Hình 42. AWS Load Balancer Controller addons bằng helm

External Secrets Operator:

```

resource "helm_release" "external_secrets" {
  name           = "external-secrets"
  repository      = "https://charts.external-secrets.io"
  chart          = "external-secrets"
  namespace      = "external-secrets"
  create_namespace = true
  version        = "0.10.0"

  depends_on = [
    module.eks,
    helm_release.aws_load_balancer_controller, # ← key fix
  ]

  wait           = true
  wait_for_jobs  = true
  timeout        = 300

  set {
    name = "installCRDs"
    value = "true"
  }

  set {
    name = "serviceAccount.create"
    value = "true"
  }

  set {
    name = "serviceAccount.name"
    value = "external-secrets"
  }

  set {
    name = "serviceAccount.annotations.eks\\.amazonaws\\.com/role-arn"
    value = module.external_secrets_irsas.iam_role_arn
  }
}

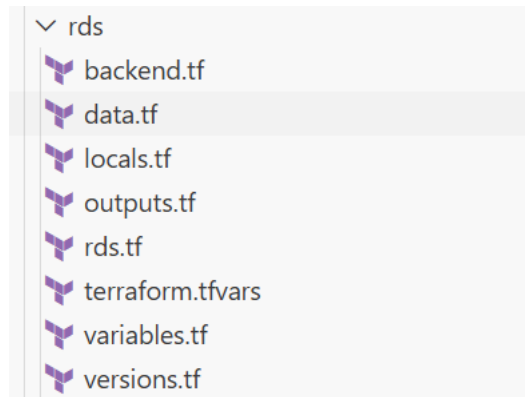
```

Hình 43. External Secrets Operator addons bằng helm

Điểm quan trọng ở đây là Terraform không chỉ tạo EKS cluster, mà còn tạo luôn một số thành phần cấp cluster bằng helm_release.

3.3.4 RDS PostgreSQL

RDS state nằm trong infrastructure/envs/rds:



Hình 44. Cấu trúc RDS state

Terraform tạo security group của RDS dựa trên output từ hai state trước: VPC ID lấy từ network state, node security group ID lấy từ EKS state.

```
resource "aws_security_group" "rds" {
  name_prefix = "${local.db_identifier}-sg-"
  description = "RDS PostgreSQL security group - only EKS workers can connect"
  vpc_id      = data.terraform_remote_state.network.outputs.vpc_id

  ingress {
    description = "PostgreSQL from EKS worker nodes"
    from_port   = var.db_port
    to_port     = var.db_port
    protocol    = "tcp"
    security_groups = [data.terraform_remote_state.eks.outputs.node_security_group_id]
  }

  # Outbound: tất cả (RDS không initiate connection ra ngoài, nhưng để default cho safety)
  egress {
    description = "Allow all outbound (RDS does not initiate connections)"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  lifecycle {
    create_before_destroy = true
  }

  tags = merge(local.common_tags, {
    Name = "${local.db_identifier}-sg"
  })
}
```

Hình 45. RDS security group

RDS có encryption at rest bằng KMS

```
resource "aws_kms_key" "rds" {
  description          = "RDS encryption key - ${local.db_identifier}"
  deletion_window_in_days = 7
  enable_key_rotation  = true

  tags = merge(local.common_tags, {
    Name = "${local.db_identifier}-encryption"
  })
}
```

Hình 46. RDS KMS encryption

RDS instance chính được tạo bằng resource `aws_db_instance`:

```

resource "aws_db_instance" "main" {
  # --- Identity ---
  identifier = local.db_identifier

  # --- Engine ---
  engine           = var.db_engine
  engine_version   = var.db_engine_version
  instance_class   = var.db_instance_class
  port             = var.db_port

  # --- Storage ---
  allocated_storage    = var.db_allocated_storage
  max_allocated_storage = var.db_max_allocated_storage
  storage_type         = "gp3"
  storage_encrypted    = true
  kms_key_id           = aws_kms_key.rds.arn

  # --- Database ---
  db_name = var.db_name
  username = var.db_username

  # --- Password - AWS-managed (Secrets Manager) ---
  # AWS auto-generate strong password, lưu Secrets Manager
  manage_master_user_password = true
  master_user_secret_kms_key_id = aws_kms_key.rds.arn

  # --- Network ---
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]
  publicly_accessible   = false

  # --- Parameter group ---
  parameter_group_name = aws_db_parameter_group.main.name

  # --- HA ---
  multi_az = var.db_multi_az

  # --- Backup ---
  backup_retention_period = var.db_backup_retention_period
  backup_window           = "03:00-04:00" # UTC = 10-11am Vietnam (low traffic)
  maintenance_window      = "Sun:04:00-Sun:05:00" # UTC

  # --- Performance Insights - DISABLED cho dev ---
  # ⚠️ Khi disabled, KHÔNG được set kms_key_id và retention_period
  # AWS RDS API sẽ reject với InvalidParameterCombination
  performance_insights_enabled = false

  # --- Enhanced Monitoring (basic CloudWatch) ---
  monitoring_interval = 60 # OS-level metrics mỗi 60s
  monitoring_role_arn = aws_iam_role.rds_monitoring.arn

  # --- CloudWatch Logs Export ---
  # PostgreSQL log types: postgresql (general), upgrade (version upgrade events)
  enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

  # --- Lifecycle ---
  skip_final_snapshot    = true # Dev: true cho tiện destroy. ⚠️ Prod: PHẢI false
  delete_automated_backups = true
  deletion_protection     = false # ⚠️ Prod: PHẢI true

  # --- Misc ---
  apply_immediately = true # Dev: true cho fast feedback. Prod: false
  copy_tags_to_snapshot = true
  auto_minor_version_upgrade = true

  tags = merge(local.common_tags, {
    Name = local.db_identifier
  })
}

```

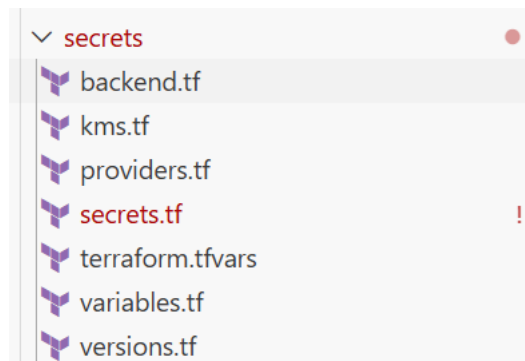
Hình 47. RDS instance

Password của master user được để AWS quản lý trong Secrets Manager

Network attachment của RDS dùng subnet group và security group vừa tạo

3.3.5 Secrets state

Secrets state nằm trong infrastructure/envs/secrets:



Hình 48. Cấu trúc Secrets state

Secrets state tạo các secret như JWT secret và Grafana admin password trong AWS Secrets Manager. RDS cũng tự quản lý master user password bằng Secrets Manager.

```
resource "random_password" "jwt_secret" {
  length = 64
  special = true

  # Avoid characters problematic in URL/JSON
  override_special = "!@#$$%^&*()-_+=[]{}"
}

resource "random_password" "jwt_refresh_secret" {
  length      = 64
  special      = true
  override_special = "!@#$$%^&*()-_+=[]{}"
}
```

Hình 49. Tạo JWT secret trong AWS Secrets Manager

```
resource "random_password" "grafana_admin" {
  length      = 32
  special      = true
  override_special = "!@#$$%^&*-_+=+"
}

resource "aws_secretsmanager_secret" "grafana" {
  name          = "${var.project}/${var.environment}/grafana"
  description    = "Grafana admin credentials"
  kms_key_id     = aws_kms_key.secrets.arn

  recovery_window_in_days = 0 # Dev: instant delete. Prod: 7-30

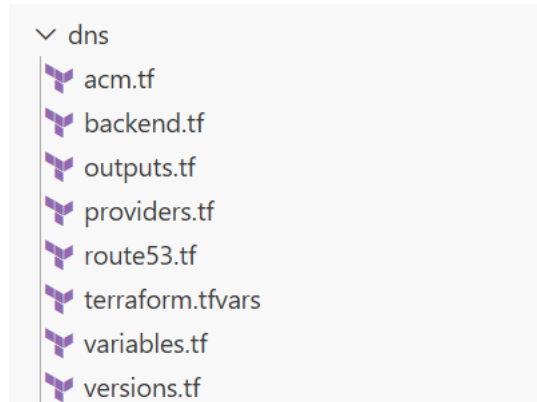
  tags = {
    Component = "monitoring"
  }
}
```

Hình 50. Tạo Grafana admin password trong AWS Secrets Manager

Kubernetes không lưu trực tiếp giá trị secret trong Git; External Secrets Operator sẽ lấy secret từ AWS và tạo Kubernetes Secret tại runtime.

3.3.6 DNS và TLS state

DNS state nằm trong infrastructure/envs/dns:



Hình 51. Cấu trúc DNS state

Với domain thật, DNS có thể gồm hai phase: phase 1 tạo certificate và validation record, phase 2 tạo A record trở về ALB sau khi ALB được tạo.

Stage 1:

DNS state dùng Route53 và ACM. ACM certificate phục vụ TLS cho ALB:

```
resource "aws_acm_certificate" "main" {
  domain_name = var.domain_name # apex domain

  subject_alternative_names = [
    "*.${var.domain_name}", # wildcard
    local.full_fqdn,        # subdomain cụ thể
  ]

  validation_method = "DNS"

  lifecycle {
    create_before_destroy = true # Tránh downtime khi rotate cert
  }

  tags = {
    Name = "${var.project}-cert"
  }
}
```

Hình 52. AWS ACM certificate phục vụ TLS cho ALB

DNS validation record được tạo từ domain_validation_options của ACM và sẽ chờ cho đến khi ACM validate xong:

```

resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.main.domain_validation_options : dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }

  allow_overwrite = true
  name            = each.value.name
  records         = [each.value.record]
  ttl             = 60
  type            = each.value.type
  zone_id         = local.zone_id
}

# — ACM cert validation (đợi DNS propagate) —
# Resource này không tạo gì, chỉ "đợi" cho ACM verify xong
resource "aws_acm_certificate_validation" "main" {
  certificate_arn      = aws_acm_certificate.main.arn
  validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]

  timeouts {
    create = "10m" # Đôi khi DNS validation lâu
  }
}

```

Hình 53. Tạo Route53 record và ACM cert validation

Stage 2:

Sau khi Kubernetes tạo ALB, Terraform lookup ALB:

```

data "aws_lb" "app" {
  count = var.alb_exists ? 1 : 0

  tags = {
    "elbv2.k8s.aws/cluster"      = local.cluster_name
    "ingress.k8s.aws/resource"    = "LoadBalancer"
    "ingress.k8s.aws/stack"      = "task-manager-dev/task-manager"
  }
}

```

Hình 54. lookup ALB sau khi tạo ALB

Và tạo Route53 alias record:

```

resource "aws_route53_record" "app" {
  count    = var.alb_exists ? 1 : 0
  zone_id = local.zone_id
  name     = local.full_fqdn
  type     = "A"

  alias {
    name            = data.aws_lb.app[0].dns_name
    zone_id         = data.aws_lb.app[0].zone_id
    evaluate_target_health = true
  }
}

```

Hình 55. Tạo Route53 alias record

3.4 Triển khai Kubernetes bằng GitOps

3.4.1 Bootstrap App-of-app Application

ArgoCD được tổ chức theo mô hình App-of-Apps. Thay vì apply từng Application riêng lẻ bằng tay, hệ thống chỉ cần apply một Application gốc, sau đó Application này sẽ quản lý các Application con.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default

  source:
    repoURL: https://github.com/Devop-Projects/nt548-config.git
    targetRevision: main
    path: platform/argocd/apps

    directory:
      recurse: true
      include: '*.yaml'

  destination:
    server: https://kubernetes.default.svc
    namespace: argocd

  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
      - ServerSideApply=true

```

Sau khi EKS sẵn sàng và ArgoCD được cài đặt, manifest thủ công quan trọng nhất là `bootstrap/root-app.yaml`. Root app trỏ đến config repo, path `platform/argocd/apps`, và dùng `directory.recurse` để đọc các Application YAML. Sau khi apply root app, các Application còn lại được quản lý qua Git.

Application	Vai trò
root	Quản lý các ArgoCD Application khác
task-manager-dev	Deploy ứng dụng Task Manager bằng Helm chart
kube-prometheus-stack	Cài Prometheus, Grafana, Alertmanager
argo-rollouts	Cài Argo Rollouts controller và CRD
monitoring-extras	Bổ sung StorageClass, ExternalSecret Grafana, Prometheus rules
grafana-dashboards	Đồng bộ dashboard backend RED

Bảng 2. Danh sách các ArgoCD Application

3.4.2 monitoring-extras Application

Application `monitoring-extras` được định nghĩa trong `platform/argocd/apps/monitoring-extras.yaml`.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: monitoring-extras
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
  annotations:
    # Sync sớm hơn các Application khác trong cùng tier
    argocd.argoproj.io/sync-wave: "-10"
spec:
  project: default

  source:
    repoURL: https://github.com/Devop-Projects/nt548-config.git
    targetRevision: main
    path: platform/monitoring
    directory:
      recurse: false
      # Chỉ pick yaml file, bỏ qua README hay file khác
      include: '*.yaml'

  destination:
    server: https://kubernetes.default.svc
    namespace: monitoring

```

Hình 56. Monitoring-extras Application

Phân tích monitoring-extras Application:

- Application này quản lý các resource phụ trợ cho monitoring stack, được lấy từ thư mục `platform/monitoring` trong config repo.
- `repoURL` trỏ về `https://github.com/Devop-Projects/nt548-config.git`, `targetRevision: main`, nghĩa là ArgoCD luôn lấy cấu hình monitoring extras từ nhánh main của config repo.
- `path: platform/monitoring` cho biết ArgoCD sẽ apply các file YAML nằm trong thư mục này.
- `directory.recurse=false` nghĩa là ArgoCD chỉ đọc trực tiếp các file trong thư mục `platform/monitoring`, không đọc sâu vào thư mục con.
- `include: '\*.yaml'` giúp ArgoCD chỉ lấy các file YAML, tránh apply nhầm README hoặc file không phải manifest.
- `destination.namespace=monitoring` nghĩa là các resource được tạo vào namespace `monitoring`.
- `sync-wave: "-10"` giúp Application này được sync sớm hơn các Application monitoring khác. Điều này cần thiết vì `monitoring-extras` chuẩn bị các resource như StorageClass hoặc ExternalSecret mà `kube-prometheus-stack` sẽ sử dụng.
- `ignoreDifferences` được cấu hình cho ExternalSecret để tránh ArgoCD báo drift giả với các field do External Secrets Operator tự thêm.
- Sync policy:
 - `prune=true` cho phép xóa resource khỏi cluster nếu resource đó bị xóa khỏi Git.
 - `selfHeal=true` giúp ArgoCD tự sửa nếu resource trong cluster bị chỉnh tay khác với Git.
 - `CreateNamespace=true` cho phép ArgoCD tạo namespace `monitoring` nếu chưa tồn tại.
 - `ServerSideApply=true` giúp apply manifest bằng cơ chế server-side, phù hợp với resource có schema phức tạp.
 - `retry.limit=5` cho phép ArgoCD tự retry khi resource phụ trợ chưa sẵn sàng ngay lần sync đầu tiên.

3.4.3 kube-prometheus-stack Application

Application kube-prometheus-stack được định nghĩa trong platform/argocd/apps/kube-prometheus-stack.yaml.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: kube-prometheus-stack
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
  annotations:
    # ⚠️ Sync SAU monitoring-extras (-10) vì cần StorageClass + Secret
    argocd.argoproj.io/sync-wave: "0"
spec:
  project: default
  # CRDs to ignore (Helm + Operator hay tạo lệch diff)
  ignoreDifferences:
    - group: apiextensions.k8s.io
      kind: CustomResourceDefinition
      jsonPointers:
        - /spec/preserveUnknownFields
    - group: admissionregistration.k8s.io
      kind: ValidatingWebhookConfiguration
      jqPathExpressions:
        - '.webhooks[?].clientConfig.caBundle'
  source:
    repoURL: https://prometheus-community.github.io/helm-charts
    chart: kube-prometheus-stack
    targetRevision: 62.7.0 # ⚠️ pin version cho reproducibility
    helm:
      releaseName: kube-prometheus-stack
      values: |
        # — Prometheus —————
        prometheus:
          prometheusSpec:
            # Retention: 15 ngày cho dev (tradeoff storage/history)
            retention: 15d
            retentionSize: "8GB"
            # Discover ServiceMonitor từ tất cả namespace
            # Mặc định nó CHỈ match release label → false sẽ catch mọi SM
            serviceMonitorSelectorNilUsesHelmValues: false
            podMonitorSelectorNilUsesHelmValues: false
            ruleSelectorNilUsesHelmValues: false
            # Storage

```

Hình 57. kube-prometheus-stack Application

Phân tích kube-prometheus-stack Application:

- Application này cài đặt bộ monitoring chính gồm Prometheus, Grafana, Alertmanager, kube-state-metrics, node-exporter và Prometheus Operator.
- Khác với `monitoring-extras`, source của Application này không lấy từ config repo nội bộ mà lấy từ Helm repository `https://prometheus-community.github.io/helm-charts`.
- `chart: kube-prometheus-stack` cho biết ArgoCD sẽ cài chart Helm chính thức của Prometheus Community.
- `targetRevision: 62.7.0` dùng để cố định version chart, tránh việc chart tự thay đổi khi upstream release phiên bản mới.
- `releaseName: kube-prometheus-stack` là tên Helm release được tạo trong cluster.
- `sync-wave: "0"` cho biết Application này sync sau `monitoring-extras`, vì `monitoring-extras` có sync wave `-10`.
- Prometheus được cấu hình `retention: 15d` và `retentionSize: 8GB`, nghĩa là dữ liệu metrics được giữ tối đa 15 ngày hoặc 8GB.

- Prometheus sử dụng PVC với `storageClassName: gp3` và dung lượng `10Gi`, vì vậy StorageClass cần được tạo trước.
- Grafana được bật với `enabled=true`, dùng `existingSecret: grafana-admin-secret` để lấy thông tin admin từ Secret có sẵn.
- Grafana bật persistence với `storageClassName: gp3` và dung lượng `2Gi`.
- Grafana Ingress dùng `ingressClassName: alb`, certificate ARN và host riêng để expose dashboard qua ALB.
- Grafana sidecar dashboards được bật, giúp Grafana tự đọc dashboard từ các ConfigMap/Secret có label phù hợp.
- `ignoreDifferences` được cấu hình cho CRD và ValidatingWebhookConfiguration để tránh drift giả do Helm hoặc Kubernetes tự cập nhật field.
- Sync policy:
 - `prune=true` cho phép ArgoCD xóa các resource monitoring không còn trong Git/Helm render output.
 - `selfHeal=true` giúp tự khôi phục cấu hình monitoring nếu bị thay đổi thủ công.
 - `allowEmpty=false` tránh trường hợp render rỗng nhưng vẫn xóa toàn bộ resource.
 - `CreateNamespace=true` cho phép tạo namespace `monitoring`.
 - `ServerSideApply=true` phù hợp với chart lớn có nhiều CRD và manifest dài.
 - `SkipDryRunOnMissingResource=true` giúp tránh lỗi dry-run trong lần cài đầu khi CRD chưa được Kubernetes API nhận diện đầy đủ.
 - `PruneLast=true` giúp xóa resource cũ sau cùng, tránh ảnh hưởng đến quá trình update.
 - `retry.limit=5` cho phép retry khi các CRD hoặc webhook của monitoring stack cần thời gian để sẵn sàng.

3.4.4 Argo-rollouts Application

Application argo-rollouts được định nghĩa trong `platform/argocd/apps/argo-rollouts.yaml`.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: argo-rollouts
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default

  ignoreDifferences:
    - group: apiextensions.k8s.io
      kind: CustomResourceDefinition
      jsonPointers:
        - /spec/preserveUnknownFields

  source:
    repoURL: https://argoproj.github.io/argo-helm
    chart: argo-rollouts
    targetRevision: 2.37.7 # Latest stable as of 2026

```

Hình 58. Argo-rollouts Application

Phân tích argo-rollouts Application:

- Application này cài Argo Rollouts controller vào cluster thông qua GitOps.
- Source trỏ đến Helm repository `https://argoproj.github.io/argo-helm`.
- `chart: argo-rollouts` cho biết ArgoCD sẽ render và cài đặt Helm chart của Argo Rollouts.
- `targetRevision: 2.37.7` cố định version chart để đảm bảo lần deploy sau vẫn dùng đúng phiên bản đã kiểm soát.
- `releaseName: argo-rollouts` là tên Helm release trong cluster.
- `installCRDs=true` cho phép chart cài các CRD cần thiết như Rollout, AnalysisTemplate và AnalysisRun.
- `controller.replicas=2` cấu hình hai replica cho Rollouts controller, giúp tăng khả năng sẵn sàng.
- `controller.metrics.enabled=true` bật metrics cho controller để có thể quan sát hoạt động của Rollouts.
- `controller.resources.requests` và `controller.resources.limits` giới hạn tài nguyên CPU/memory cho controller.
- Dashboard của Argo Rollouts được bật với `dashboard.enabled=true`.
- Dashboard dùng Service `ClusterIP`, nghĩa là chỉ truy cập nội bộ hoặc thông qua port-forward, không expose trực tiếp ra Internet.
- `ignoreDifferences` được cấu hình cho CRD để tránh ArgoCD báo khác biệt giả ở field `preserveUnknownFields`.
- Sync policy:
 - `prune=true` giúp xóa resource của Argo Rollouts nếu không còn trong chart.
 - `selfHeal=true` giúp khôi phục controller nếu cấu hình bị chỉnh tay.
 - `CreateNamespace=true` cho phép tạo namespace `argo-rollouts`.
 - `ServerSideApply=true` giúp apply CRD và resource lớn ổn định hơn.

3.4.5 Grafana-dashboards Application

Application grafana-dashboards được định nghĩa trong platform/argocd/apps/grafana-dashboards.yaml.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: grafana-dashboards
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default

  source:
    repoURL: https://github.com/Devop-Projects/nt548-config.git
    targetRevision: main
    path: platform/monitoring/dashboards

  destination:
    server: https://kubernetes.default.svc
    namespace: monitoring

  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - ServerSideApply=true
      - CreateNamespace=false
    retry:
      limit: 3
      backoff:
        duration: 30s
        factor: 2
        maxDuration: 3m
```

Hình 59. grafana-dashboards Application

Phân tích grafana-dashboards Application:

- Application này dùng để đồng bộ dashboard Grafana từ Git vào cluster.
- Source trỏ về config repo `https://github.com/Devop-Projects/nt548-config.git`.
- `targetRevision: main` nghĩa là ArgoCD lấy dashboard từ nhánh main.
- `path: platform/monitoring/dashboards` cho biết thư mục chứa các manifest dashboard.
- `destination.namespace=monitoring` nghĩa là dashboard được apply vào namespace `monitoring`.
- Application này không dùng Helm chart mà apply trực tiếp các YAML trong thư mục dashboard.

- Cách làm này biến dashboard thành “dashboard-as-code”: dashboard được lưu trong Git, có thể review, rollback và đồng bộ tự động.
- `CreateNamespace=false` vì namespace `monitoring` đã được tạo bởi các Application monitoring trước đó.
- Sync policy:
 - `prune=true` giúp xóa dashboard khỏi cluster nếu file dashboard bị xóa khỏi Git.
 - `selfHeal=true` giúp ArgoCD khôi phục dashboard nếu có chỉnh sửa thủ công trong cluster.
 - `ServerSideApply=true` giúp apply manifest dashboard ổn định hơn.
 - `retry.limit=3` cho phép retry khi monitoring namespace hoặc Grafana sidecar chưa sẵn sàng ngay lúc đầu.

3.4.6 Task-manager-dev Application

Application chính của hệ thống là task-manager-dev, được định nghĩa trong platform/argocd/apps/task-manager-dev.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: task-manager-dev
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default

  source:
    repoURL: https://github.com/Devop-Projects/nt548-config.git
    targetRevision: main
    path: charts/task-manager
    helm:
      releaseName: task-manager
      valueFiles:
        - values.yaml
        - values-aws-dev.yaml

  destination:
    server: https://kubernetes.default.svc
    namespace: task-manager-dev
```

Hình 60. Task-manager-dev Application

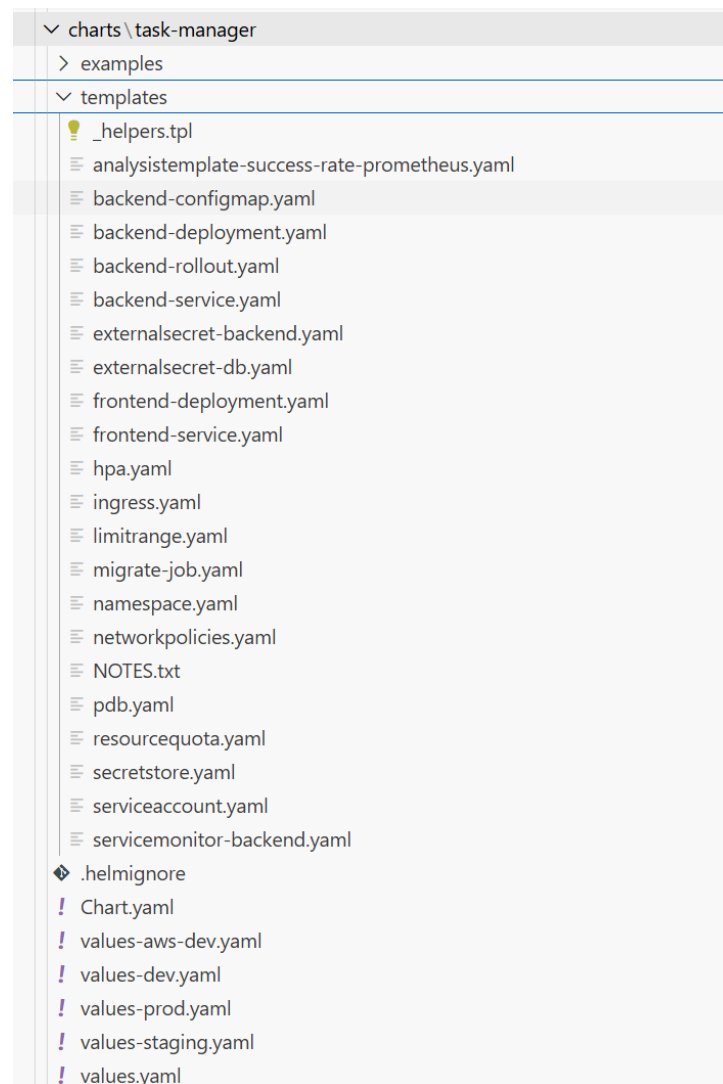
Phân tích Task-manager-dev Application:

- ArgoCD deploy ứng dụng bằng Helm chart nội bộ tại charts/task-manager
- File values.yaml chứa cấu hình mặc định, còn values-aws-dev.yaml ghi đè cấu hình riêng cho môi trường AWS dev. Nhờ vậy, cùng một chart có thể dùng lại cho nhiều môi trường khác nhau.
- Sync policy:

- CreateNamespace=true cho phép ArgoCD tự tạo namespace nếu chưa có.
- PruneLast=true giúp ArgoCD xóa resource cũ ở cuối quá trình sync, tránh xóa quá sớm gây gián đoạn.
- ApplyOutOfSyncOnly=true giúp chỉ apply các resource đang lệch trạng thái, giảm thao tác không cần thiết.
- retry.limit: 10 phù hợp với môi trường có ExternalSecret, Job migration hoặc CRD cần thời gian để sẵn sàng.

3.5 Task Manager Helm chart

Helm chart task-manager là toàn bộ workload ứng dụng chạy trên Kubernetes. Chart này nằm trong nt548-config/charts/task-manager:



Hình 61. Task Manager Helm chart

chart quản lý hai thành phần chính là backend và frontend.

- Backend được cấu hình image, replica, service port, biến môi trường, probes và security context thông qua nhóm backend trong values.

```

backend:
  enabled: true
  replicaCount: 2

  image:
    repository: doanvantai/nt548-backend
    tag: "8calfeb"          # CI sẽ update field này
    pullPolicy: IfNotPresent

  service:
    type: ClusterIP
    port: 3000
    targetPort: http

  # Pod-level security context
  podSecurityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 1000
    fsGroup: 1000

  # Container-level security context
  containerSecurityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities:
      drop: ["ALL"]
    seccompProfile:
      type: RuntimeDefault

```

Hình 62. Backend helm values

- Frontend được cấu hình tương tự qua nhóm frontend trong values.

```

frontend:
  enabled: true
  replicaCount: 2

  image:
    repository: doanvantai/nt548-frontend
    tag: "8calfeb"
    pullPolicy: IfNotPresent

  service:
    type: ClusterIP
    port: 80
    targetPort: 8080

  # Nginx Alpine runs as UID 101
  podSecurityContext:
    runAsNonRoot: true
    runAsUser: 101
    runAsGroup: 101
    fsGroup: 101

  containerSecurityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities:
      drop: ["ALL"]
    seccompProfile:
      type: RuntimeDefault

```

Hình 63. Frontend helm values

Backend khả năng render theo hai kiểu: Deployment hoặc Rollout của Argo Rollouts. Trong cấu hình mặc định, backend.rollout.enabled=false, backend dùng Deployment thông thường. Trong values-aws-dev.yaml, giá trị này là true

```

rollout:
  enabled: false

  strategy: "blueGreen"

  analysisTemplates:
    # Master switch – tắt = không render AnalysisTemplate
    enabled: false

# — Blue-Green config —
blueGreen:
  # Service names – sẽ được Argo Rollouts manage
  activeService: backend
  previewService: backend-preview

  # Auto-promote? false = manual, true = tự promote sau analysis pass
  autoPromotionEnabled: false

  # Hoặc auto-promote sau N giây (override autoPromotionEnabled)
  # autoPromotionSeconds: 600

  # Đợi bao lâu trước khi scale down phiên bản cũ
  scaleDownDelaySeconds: 30

# Pre-promotion analysis (trước khi switch)
prePromotionAnalysis:
  enabled: false          # Bật khi có AnalysisTemplate ready
  templates:
    - success-rate-web

# Post-promotion analysis (sau khi switch)
postPromotionAnalysis:
  enabled: false
  templates:
    - success-rate-web

```

Hình 64. Backend Rollout helm values

Đối với các biến không nhạy cảm, ứng dụng có thể lưu trực tiếp trên values.yaml, còn đối với các secret thì sẽ được External Secrets Operator đồng bộ dữ liệu để tự động tạo Kubernetes Secret để Pod sử dụng.

```

# External Secret config – chỉ enable trên AWS
externalSecret:
  enabled: false
  refreshInterval: 1h
  secretStoreName: aws-secrets-manager
  awsSecretName: ""          # Override: "devops/dev/backend/secrets"
  dbSecretName: ""          # Override: "rds!db-xxxx-yyyy"

```

Hình 65. Secrets helm values

3.6 Quy trình Rollout backend

Trong môi trường AWS dev, backend không dùng Deployment thường mà dùng Argo Rollouts. Điều này được bật trong values-aws-dev.yaml:

```
rollout:
  enabled: true
  strategy: blueGreen
  analysisTemplates:
    enabled: true
  blueGreen:
    activeService: backend
    previewService: backend-preview
    autoPromotionEnabled: false
    scaleDownDelaySeconds: 30
    prePromotionAnalysis:
      enabled: true
      templates:
        - success-rate-prometheus
    postPromotionAnalysis:
      enabled: false
      templates: []
```

Hình 66. Backend rollout aws values

Quy trình Rollout:

- Workflow triển khai backend diễn ra theo kiểu blue/green deployment.
- Khi CI cập nhật image tag mới trong values-aws-dev.yaml, ArgoCD sẽ sync Helm chart và render ra resource Rollout.
- Argo Rollouts controller phát hiện Pod template thay đổi, sau đó tạo ReplicaSet mới cho phiên bản mới của backend.
- Backend phiên bản mới sẽ được trỏ tới service có label role: preview
- Lúc này traffic vẫn đi qua backend phiên bản cũ do ingress chỉ truyền traffic đến service active
- Để phiên bản mới được cập nhật, Rollout chạy pre-promotion analysis có template là success-rate-prometheus
- AnalysisTemplate success-rate-prometheus dùng Prometheus để theo dõi bản preview, các metrics gồm:

```

metrics:
  # Basic scrape health. This is safe to test even when preview has no user traffic.
  - name: prometheus-target-up
    interval: 30s
    count: 5
    initialDelay: 30s
    successCondition: result[0] >= 1
    failureLimit: 2
    provider:
      prometheus:
        address: http://kube-prometheus-stack-prometheus.monitoring.svc.cluster.local:9090
        query: |
          max(
            up{
              namespace="{{`{{args.namespace}}`}}",
              service="{{`{{args.service-name}}`}}"
            }
          )

```

Hình 67. metric prometheus-target-up

- prometheus-target-up: kiểm tra service preview có được Prometheus scrape hay không.

```

# NOTE: DASHBOARD METRICS USE STATUS_CODE, NOT STATUS.
- name: http-success-rate
  interval: 30s
  count: 5
  initialDelay: 60s
  successCondition: result[0] >= 0.99
  failureLimit: 2
  provider:
    prometheus:
      address: http://kube-prometheus-stack-prometheus.monitoring.svc.cluster.local:9090
      query: |
        (
          sum(rate(http_requests_total{
            namespace="{{`{{args.namespace}}`}}",
            service="{{`{{args.service-name}}`}}",
            status_code!~"5.."
          }[2m]))
          /
          clamp_min(
            sum(rate(http_requests_total{
              namespace="{{`{{args.namespace}}`}}",
              service="{{`{{args.service-name}}`}}"
            }[2m])),
            0.001
          )
        )

```

Hình 68. Metric http-success-rate

- http-success-rate: kiểm tra tỷ lệ request không lỗi 5xx, yêu cầu đạt ít nhất 0.99

```

- name: p99-latency
  interval: 30s
  count: 5
  initialDelay: 60s
  successCondition: result[0] < 0.5
  failureLimit: 2
  provider:
    prometheus:
      address: http://kube-prometheus-stack-prometheus.monitoring.svc.cluster.local:9090
      query: |
        histogram_quantile(
          0.99,
          sum(rate(http_request_duration_seconds_bucket{
            namespace="{{`{{args.namespace}}`}}",
            service="{{`{{args.service-name}}`}}"
          }[2m])) by (le)
        )

```

Hình 69.metric p99-latency

- p99-latency: kiểm tra độ trễ p99 nhỏ hơn 0.5 giây.

Kịch bản promote:

Khi phiên bản mới đã được sync và analysisRun ở trạng thái healthy, ta có thể promote phiên bản mới bằng lệnh:

```
kubectl argo rollouts promote backend -n task-manager-dev
```

Sau khi promote, Argo Rollouts chuyển active service backend từ ReplicaSet cũ sang ReplicaSet mới. Từ thời điểm này, traffic thật đi qua Ingress /api sẽ vào phiên bản backend mới. Phiên bản cũ chưa bị xóa ngay mà được giữ trong một khoảng ngắn, giúp quá trình chuyển đổi an toàn hơn.

Kịch bản rollback hoặc abort:

Nếu phiên bản mới lỗi trước khi promote, ví dụ Pod không ready, readiness probe fail hoặc analysisRun ở trạng thái degraded, có thể abort rollout:

```
kubectl argo rollouts abort backend -n task-manager-dev
```

Trong trường hợp này, active service backend vẫn trỏ đến phiên bản cũ, nên traffic thật chưa bị ảnh hưởng. Phiên bản preview mới sẽ bị dừng hoặc scale down tùy trạng thái rollout.

Nếu phiên bản mới đã được promote nhưng sau đó phát hiện lỗi, có thể rollback về revision trước:

```
kubectl argo rollouts undo backend -n task-manager-dev
```

Khi rollback, Argo Rollouts tạo lại trạng thái mong muốn dựa trên revision cũ và chuyển service về phiên bản ổn định trước đó.

3.7 Tự động hóa vận hành

Tự động hóa vận hành của dự án được xây dựng chủ yếu bằng Makefile trong repo NT548-DevOps. Makefile đóng vai trò như một runbook tự động: thay vì người vận hành phải nhớ nhiều lệnh Terraform, AWS CLI, kubectl, Git và ArgoCD, các thao tác được gom thành các target như make deploy, make status, make verify, make hibernate, make wake.

3.7.1 Cấu hình vận hành tập trung

Makefile khai báo các biến dùng chung ở đầu file:

```
# — Project config —————
PROJECT      := devops
ENVIRONMENT  := dev
REGION       := ap-southeast-1
NAMESPACE    := task-manager-dev
CLUSTER_NAME := $(PROJECT)-$(ENVIRONMENT)

# — Paths —————
INFRA_DIR    := infrastructure
ENVS_DIR     := $(INFRA_DIR)/envs

# Config repo - chứa Helm chart + ArgoCD app definitions
# ⚠ Set CONFIG_REPO env var nếu repo không ở $HOME/nt548-config
CONFIG_REPO  ?= $(HOME)/nt548-config

# ArgoCD
ARGOCD_NAMESPACE := argocd
ARGOCD_VERSION   := stable
```

Hình 70. Cấu hình Makefile tập trung

Các biến này giúp toàn bộ workflow dùng cùng một cấu hình. Ví dụ tên cluster được tạo từ PROJECT và ENVIRONMENT, namespace ứng dụng là task-manager-dev, region AWS là ap-southeast-1, còn config repo mặc định nằm ở \$HOME/nt548-config.

Nhờ cách gom biến như vậy, nếu cần đổi môi trường hoặc đổi đường dẫn config repo, người vận hành không phải sửa từng lệnh riêng lẻ.

3.7.2 Preflight check trước khi deploy

Trước khi chạy deploy, Makefile chạy target preflight để kiểm tra môi trường vận hành. preflight:

```

..
.PHONY: preflight
preflight: ## Check tools, AWS credentials, config repo
@echo ""
@echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE) Preflight Checks$(COLOR_RESET)"
@echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE)► Checking required tools...$(COLOR_RESET)"
@command -v aws >/dev/null 2>&1 || { echo "$(COLOR_RED) X aws cli NOT found$(COLOR_RESET)"; \
@echo " → aws cli: $$ (aws --version 2>&1 | head -1)"; \
@command -v terraform >/dev/null 2>&1 || { echo "$(COLOR_RED) X terraform NOT found$(COLOR_RESET)"; \
@echo " → terraform: $$ (terraform version | head -1)"; \
@command -v kubectl >/dev/null 2>&1 || { echo "$(COLOR_RED) X kubectl NOT found$(COLOR_RESET)"; \
@echo " → kubectl: $$ (kubectl version --client 2>&1 | grep -i 'client version' | head -1)"; \
@command -v yq >/dev/null 2>&1 || { \
echo "$(COLOR_RED) X yq NOT found. Install:$(COLOR_RESET)"; \
echo "$(COLOR_YELLOW) sudo wget -qO /usr/local/bin/yq https://github.com/mikefarah/yq/"; \
echo "$(COLOR_YELLOW) sudo chmod +x /usr/local/bin/yq$(COLOR_RESET)"; \
exit 1; }
@echo " → yq: $$ (yq --version)"
@echo "$(COLOR_GREEN) ✓ All tools available$(COLOR_RESET)"
@echo ""
@echo "$(COLOR_BLUE)► Checking AWS credentials...$(COLOR_RESET)"
@aws sts get-caller-identity >/dev/null 2>&1 || { \
echo "$(COLOR_RED) X AWS credentials invalid or expired$(COLOR_RESET)"; \
echo "$(COLOR_YELLOW) Run: aws configure$(COLOR_RESET)"; \
exit 1; }
@echo " → Account: $$ (aws sts get-caller-identity --query Account --output text)"
@echo " → User: $$ (aws sts get-caller-identity --query Arn --output text)"
@echo " → Region: $(REGION)"
@echo "$(COLOR_GREEN) ✓ AWS credentials valid$(COLOR_RESET)"
@echo ""
@echo "$(COLOR_BLUE)► Checking config repo...$(COLOR_RESET)"
@[ -d "$(CONFIG_REPO)" ] || { \
echo "$(COLOR_RED) X Config repo NOT found at $(CONFIG_REPO)$(COLOR_RESET)"; \
echo "$(COLOR_YELLOW) Either clone the config repo to $(HOME)/nt548-config$(COLOR_RESET)"; \
echo "$(COLOR_YELLOW) Or: export CONFIG_REPO=/path/to/your/nt548-config$(COLOR_RESET)"; \
exit 1; }

```

Hình 71. Lệnh Make preflight

Preflight kiểm tra các công cụ bắt buộc gồm AWS CLI, Terraform, kubectl và yq. Nếu thiếu một công cụ, workflow dừng sớm thay vì chạy đến giữa chừng mới lỗi.

Makefile cũng kiểm tra AWS credential xác nhận người vận hành đã cấu hình AWS credential hợp lệ. Ngoài ra, Makefile còn kiểm tra config repo có tồn tại và có đủ hai phần quan trọng:

Điều này đảm bảo repo GitOps đã được clone đúng và có Helm chart cùng ArgoCD Application definitions trước khi deploy.

3.7.3 Đồng bộ config repo trước khi thao tác

Target sync-config-repo dùng để kéo trạng thái mới nhất của repo nt548-config trước khi deploy hoặc cập nhật values.

```

sync-config-repo: ## Pull/rebase local nt548-config từ origin/main
@echo ""
@echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE) Sync Config Repo$(COLOR_RESET)"
@echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
@[ -d "$(CONFIG_REPO)/.git" ] || { \
    echo "$(COLOR_RED) X $(CONFIG_REPO) is not a git repo$(COLOR_RESET)"; \
    echo "$(COLOR_YELLOW) Clone config repo first: git clone <url> $(CONFIG_REPO)$(COLOR_RESET)"; \
    exit 1; }
@cd "$(CONFIG_REPO)" && \
    export GIT_PAGER=cat; \
    BRANCH=$(git rev-parse --abbrev-ref HEAD); \
    if [ "$BRANCH" != "main" ]; then \
        echo "$(COLOR_RED) X Config repo must be on branch main, currently: $$BRANCH$(COLOR_RESET)"; \
        echo "$(COLOR_YELLOW) Run: cd $(CONFIG_REPO) && git switch main$(COLOR_RESET)"; \
        exit 1; \
    fi; \
    if [ -d .git/rebase-merge ] || [ -d .git/rebase-apply ]; then \
        echo "$(COLOR_RED) X Config repo has an unfinished rebase$(COLOR_RESET)"; \
        echo "$(COLOR_YELLOW) Resolve it in $(CONFIG_REPO), then rerun make sync-config-repo$(COLOR_RESET)"; \
        exit 1; \
    fi; \
    if [ -n "$(git status --porcelain)" ]; then \
        echo "$(COLOR_RED) X Config repo has uncommitted changes$(COLOR_RESET)"; \
        git --no-pager status --short; \
        echo "$(COLOR_YELLOW) Commit/stash these changes before syncing to avoid overwriting$(COLOR_RESET)"; \
        exit 1; \
    fi; \
    echo "$(COLOR_BLUE)▶ Fetching origin/main...$(COLOR_RESET)"; \
    git fetch origin main --quiet; \
    BEFORE=$(git rev-parse --short HEAD); \
    git pull --rebase origin main; \
    AFTER=$(git rev-parse --short HEAD); \
    if [ "$BEFORE" = "$AFTER" ]; then \
        echo "$(COLOR_GREEN) ✓ Config repo already up to date ($AFTER)$(COLOR_RESET)"; \
    else \
        echo "$(COLOR_GREEN) ✓ Config repo synced: $$BEFORE → $$AFTER$(COLOR_RESET)"; \
    fi
@echo ""

```

Hình 72. Lệnh make sync-config-repo

Target này kiểm tra config repo phải ở branch main, không có rebase đang dở và không có thay đổi chưa commit. Sau đó mới fetch và pull --rebase.

Cách làm này giúp tránh ghi đè thay đổi thủ công trong config repo. Đây là điểm quan trọng vì config repo là nguồn sự thật của GitOps; nếu repo local bị lệch hoặc có file đang sửa dở, việc automation tự commit có thể gây xung đột hoặc ghi sai cấu hình.

3.7.4 Bootstrap tạo S3 và DynamoDB cho terraform state

```

.PHONY: bootstrap
bootstrap: ## Tạo S3 + DynamoDB cho Terraform state (CHẠY 1 LẦN)
    @echo ""
    @echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_BLUE) Bootstrap Terraform Backend$(COLOR_RESET)"
    @echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_YELLOW) ⚠ Chỉ chạy lệnh này MỘT LẦN duy nhất trong đời project$(COLOR_RESET)"
    @echo "$(COLOR_YELLOW) ⚠ Nó tạo S3 bucket + DynamoDB lock table cho TF state$(COLOR_RESET)"
    @echo ""
    @echo "$(COLOR_BLUE)► Initializing Terraform...$(COLOR_RESET)"
    @cd $(INFRA_DIR)/bootstrap && terraform init
    @echo ""
    @echo "$(COLOR_BLUE)► Applying bootstrap resources...$(COLOR_RESET)"
    @cd $(INFRA_DIR)/bootstrap && terraform apply -auto-approve
    @echo ""
    @echo "$(COLOR_GREEN)$(COLOR_BOLD)✓ Bootstrap complete$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Bạn không cần chạy lệnh 'make bootstrap' nữa.$(COLOR_RESET)"
    @echo ""

```

Hình 73. Makefile bootstrap

Đây là lệnh chỉ chạy 1 lần để tạo S3 bucket và dynamoDB để lưu trữ terraform state cho toàn bộ hạ tầng

3.7.5 Deploy tổng thể từ zero đến full system

Target chính là make deploy.

```

.PHONY: deploy
deploy: preflight ## Deploy full system từ zero (~35 phút)
    @SECONDS=0; \
    echo ""; \
    echo "$(COLOR_BOLD)$(COLOR_BLUE) DEPLOY: zero → full system $(COLOR_RESET)"; \
    echo "$(COLOR_BOLD)$(COLOR_BLUE) Estimated time: ~35 minutes $(COLOR_RESET)"; \
    echo ""; \
    echo "$(COLOR_CYAN)Stages:$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 1. Terraform Infrastructure (~22 min)$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 2. DNS Phase 1: ACM Certificate (~3 min)$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 3. Sync Helm values with TF outputs (~30s)$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 4. ArgoCD + Apps Bootstrap (~10 min)$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 5. Wait ALBs ready (~3 min)$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 6. DNS Phase 2: A records (~1 min)$(COLOR_RESET)"; \
    echo "$(COLOR_CYAN) 7. Verify end-to-end$(COLOR_RESET)"; \
    echo ""; \
    $(MAKE) sync-config-repo || { echo "$(COLOR_RED)X Config repo sync failed$(COLOR_RESET)"; \
    $(MAKE) _stage-1-infrastructure || { echo "$(COLOR_RED)X Stage 1 failed$(COLOR_RESET)"; \
    $(MAKE) _stage-2-dns-phase1 || { echo "$(COLOR_RED)X Stage 2 failed$(COLOR_RESET)"; \
    $(MAKE) _stage-3-sync-helm-values || { echo "$(COLOR_RED)X Stage 3 failed$(COLOR_RESET)"; \
    $(MAKE) _stage-4-argocd || { echo "$(COLOR_RED)X Stage 4 failed$(COLOR_RESET)"; \
    $(MAKE) _stage-5-wait-alb || { echo "$(COLOR_RED)X Stage 5 failed$(COLOR_RESET)"; \
    $(MAKE) _stage-6-dns-phase2 || { echo "$(COLOR_RED)X Stage 6 failed$(COLOR_RESET)"; \
    $(MAKE) _stage-7-verify || { echo "$(COLOR_RED)X Stage 7 failed$(COLOR_RESET)"; \
    DURATION=$((SECONDS); \

```

Hình 74. Makefile deploy

Workflow deploy được chia thành 7 stage:

- Stage 1: tạo hạ tầng bằng Terraform.
- Stage 2: tạo ACM certificate và DNS validation record.
- Stage 3: đồng bộ Terraform output vào Helm values.
- Stage 4: cài ArgoCD và bootstrap các Application.
- Stage 5: chờ ALB được tạo.
- Stage 6: tạo Route53 alias record trỏ domain về ALB.
- Stage 7: verify end-to-end.

Cách chia stage này phản ánh đúng phụ thuộc của hệ thống. Ví dụ, không thể tạo Route53 alias record ngay từ đầu vì ALB chỉ xuất hiện sau khi Kubernetes Ingress được AWS Load Balancer Controller xử lý. Vì vậy DNS được chia thành hai phase: phase 1 tạo certificate trước, phase 2 tạo A record sau khi ALB tồn tại.

3.7.6 Stage 1: tự động hóa Terraform infrastructure

Trong stage `_stage-1-infrastructure`, Makefile khởi tạo lần lượt các Terraform state:

```

.PHONY: _stage-1-infrastructure
_stage-1-infrastructure:
    @echo ""
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE) Stage 1/7: Terraform Infrastructure (~22 min) $(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo ""
    @$(MAKE) setup-symlinks
    @echo "$(COLOR_BLUE)► [1/5] Initializing all Terraform states...$(COLOR_RESET)"
    @echo "  $(COLOR_GRAY)→ network state$(COLOR_RESET)"
    @cd $(ENVS_DIR)/dev && terraform init -reconfigure \
        -backend-config=../../backend-config.hcl \
        -backend-config="key=$(ENVIRONMENT)/network/terraform.tfstate" > /dev/null
    @echo "  $(COLOR_GRAY)→ eks state$(COLOR_RESET)"
    @cd $(ENVS_DIR)/eks && terraform init -reconfigure \
        -backend-config=../../backend-config.hcl \
        -backend-config="key=$(ENVIRONMENT)/eks/terraform.tfstate" > /dev/null
    @echo "  $(COLOR_GRAY)→ rds state$(COLOR_RESET)"
    @cd $(ENVS_DIR)/rds && terraform init -reconfigure \
        -backend-config=../../backend-config.hcl \
        -backend-config="key=$(ENVIRONMENT)/rds/terraform.tfstate" > /dev/null
    @echo "  $(COLOR_GRAY)→ secrets state$(COLOR_RESET)"
    @cd $(ENVS_DIR)/secrets && terraform init -reconfigure \
        -backend-config=../../backend-config.hcl \
        -backend-config="key=$(ENVIRONMENT)/secrets/terraform.tfstate" > /dev/null
    @echo "  $(COLOR_GRAY)→ dns state$(COLOR_RESET)"
    @cd $(ENVS_DIR)/dns && terraform init -reconfigure \
        -backend-config=../../backend-config.hcl \
        -backend-config="key=$(ENVIRONMENT)/dns/terraform.tfstate" > /dev/null
    @echo "$(COLOR_GREEN) ✓ All 5 states initialized$(COLOR_RESET)"
    @echo ""
    @echo "$(COLOR_BLUE)► [2/5] Applying network state (~2 min)...$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Creating: VPC, subnets, NAT gateway, route tables$(COLOR_RESET)"
    @cd $(ENVS_DIR)/dev && terraform apply -auto-approve
    @echo "$(COLOR_GREEN) ✓ Network ready$(COLOR_RESET)"
    @echo ""
    @echo "$(COLOR_BLUE)► [3/5] Applying EKS state (~15 min, đi pha cà phê)...$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Creating in order:$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) 1. EKS control plane (~10 min. AWS managed)$(COLOR_RESET)"

```

Hình 75. Stage 1 makefile infrastructure

Cách này được lặp lại cho tất cả các state:

Sau khi init, Makefile apply từng phần theo thứ tự: network, EKS, RDS, secrets. Thứ tự này cần thiết vì EKS cần network trước, RDS cần network và security group từ EKS, còn workload sau này cần secrets để chạy.

Sau khi EKS được tạo, Makefile tự cấu hình kubectl

Nhờ đó, sau khi Terraform tạo cluster, các lệnh kubectl ở stage sau có thể làm việc ngay với cluster mới.

3.7.7 Đồng bộ Terraform output vào Helm values

Sau khi RDS, Secrets và DNS phase 1 được tạo, một số giá trị thực tế cần được đưa vào config repo. Ví dụ RDS endpoint, RDS secret ARN và ACM certificate ARN có thể thay đổi sau mỗi lần recreate.


```

.PHONY: _stage-3-sync-helm-values
_stage-3-sync-helm-values:
    @echo ""
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE) Stage 3/7: Sync Helm Values với TF outputs$(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Why? RDS endpoint, ACM cert ARN, RDS secret name đổi mỗi lần recre"
    @echo "$(COLOR_CYAN) Script này đọc TF outputs → update file Helm values trong config r"
    @echo "$(COLOR_CYAN) Sau đó commit + push → ArgoCD sẽ pick up$(COLOR_RESET)"
    @echo ""
    @$ (MAKE) sync-config-repo
    @echo "$(COLOR_BLUE)▶ Running update-helm-values.sh --commit...$(COLOR_RESET)"
    @./scripts/update-helm-values.sh --commit
    @echo "$(COLOR_GREEN) ✓ Config repo updated and pushed$(COLOR_RESET)"
    @echo ""
    @echo "$(COLOR_GREEN)$(COLOR_BOLD)✓ Stage 3 complete$(COLOR_RESET)"

```

Hình 76. Stage 3 makefle sync helm values

Script update-helm-values.sh đọc output từ Terraform, sau đó dùng yq để cập nhật config repo

Script cũng cập nhật certificate ARN cho Grafana trong kube-prometheus-stack.yaml:

Cuối cùng, nếu có thay đổi, script commit và push lên config repo

Đây là một điểm quan trọng trong workflow vận hành. Terraform không ghi trực tiếp vào Kubernetes manifest đang chạy, mà output của Terraform được đồng bộ vào GitOps repo. Sau đó ArgoCD đọc config repo để đưa cấu hình mới vào cluster. Cách này vẫn giữ Git là nguồn sự thật.

3.7.8 Cài ArgoCD và bootstrap Application

Stage _stage-4-argocd gồm hai phần:

```

.PHONY: _stage-4-argocd
_stage-4-argocd:
    @echo ""
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE) Stage 4/7: ArgoCD + Apps Bootstrap (~10 min)$(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @$ (MAKE) _install-argocd
    @$ (MAKE) _bootstrap-apps
    @echo ""
    @echo "$(COLOR_GREEN)$(COLOR_BOLD)✓ Stage 4 complete$(COLOR_RESET)"

```

Hình 77. Stage 4 makefle argocd

Phần _install-argocd tạo namespace và apply manifest chính thức của ArgoCD:

```
.PHONY: _install-argocd
_install-argocd:
    @echo ""
    @echo "$(COLOR_BLUE)► [1/2] Installing ArgoCD...$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Note: ArgoCD doesn't manage itself (chicken-and-egg)$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Installing from official manifest with --server-side$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) (--server-side avoids 'annotation too long' error for large CRDs)$(COLOR_RESET)"
    @kubectl get namespace $(ARGOCD_NAMESPACE) >/dev/null 2>&1 || { \
        echo " $(COLOR_GRAY)→ Creating namespace argocd$(COLOR_RESET)"; \
        kubectl create namespace $(ARGOCD_NAMESPACE); \
    }
    @echo " $(COLOR_GRAY)→ Applying ArgoCD manifests (server-side)...$(COLOR_RESET)"
    @kubectl apply -n $(ARGOCD_NAMESPACE) \
        -f https://raw.githubusercontent.com/argoproj/argo-cd/$(ARGOCD_VERSION)/manifests/install.yaml \
        --server-side >/dev/null
    @echo "$(COLOR_BLUE)► Waiting for ArgoCD deployments (~3 min)...$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Deployments: argocd-server, repo-server, applicationset, dex, notifications$(COLOR_RESET)"
    @kubectl wait --for=condition=Available deployment --all \
        -n $(ARGOCD_NAMESPACE) --timeout=300s
    @POD_COUNT=$(kubectl get pods -n $(ARGOCD_NAMESPACE) --no-headers | grep -c Running); \
    echo "$(COLOR_GREEN) ✓ ArgoCD installed ($(POD_COUNT) pods Running)$(COLOR_RESET)"
```

Hình 78. Makefile install-argocd

Sau đó Makefile chờ tất cả deployment của ArgoCD sẵn sàng

Phần `_bootstrap-apps` triển khai các ArgoCD Application theo thứ tự. Trước tiên, Makefile kiểm tra External Secrets Operator:

```
.PHONY: _bootstrap-apps
_bootstrap-apps:
    @echo ""
    @echo "$(COLOR_BLUE)► [2/2] Bootstrapping ArgoCD applications...$(COLOR_RESET)"
    @$(MAKE) sync-config-repo
    @echo ""
    @echo "$(COLOR_BLUE)► Pre-check: External Secrets Operator must be ready$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Why? ESO installed by Terraform (not ArgoCD). ArgoCD apps depend on it.$(COLOR_RESET)"
    @kubectl wait --for=condition=Available deployment/external-secrets \
        -n external-secrets --timeout=180s
    @kubectl get crd externalsecrets.external-secrets.io >/dev/null
    @echo "$(COLOR_GREEN) ✓ ESO ready, CRDs installed$(COLOR_RESET)"
    @echo ""
    @echo "$(COLOR_BLUE)► Pre-step: Apply ClusterSecretStore BEFORE monitoring apps$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) Why? grafana-admin-secret ExternalSecret needs ClusterSecretStore$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) to exist BEFORE monitoring-extras syncs. Otherwise Health=Degraded.$(COLOR_RESET)"
    @kubectl apply -f $(CONFIG_REPO)/apps/task-manager/overlays/dev/secret-store.yaml >/dev/null || true
    @echo " $(COLOR_GRAY)→ Verifying ClusterSecretStore is ready...$(COLOR_RESET)"
    @for i in $(seq 1 12); do \
        STATUS=$(kubectl get clustersecretstore aws-secrets-manager \
            -o jsonpath='{.status.conditions[?(@.type=="Ready")].status}' 2>/dev/null || echo "Unknown"); \
        if [ "$STATUS" = "True" ]; then \
            echo "$(COLOR_GREEN) ✓ ClusterSecretStore ready$(COLOR_RESET)"; break; \
        fi; \
        echo " [Attempt $$i/12] ClusterSecretStore status: $STATUS, waiting 10s..."; \
        sleep 10; \
    done
```

Hình 79. Makefile bootstrap-apps

Sau đó apply ClusterSecretStore trước monitoring app, vì Grafana admin secret cần SecretStore để ExternalSecret hoạt động. Tiếp theo Makefile apply các Application nền. Cuối cùng mới apply ứng dụng chính.

Nhờ đó, automation không chỉ apply manifest rồi kết thúc, mà thật sự đợi ArgoCD đồng bộ xong trước khi chuyển sang bước tiếp theo.

3.7.9 Chờ ALB và cập nhật DNS phase 2

Sau khi ArgoCD sync ứng dụng và monitoring, AWS Load Balancer Controller sẽ tạo ALB từ Ingress. ALB không xuất hiện ngay lập tức, nên Makefile có stage chờ ALB:


```

.PHONY: _stage-5-wait-alb
_stage-5-wait-alb:
    @echo ""
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE) Stage 5/7: Wait ALBs ready (~3 min)$(COLOR_RESET)"
    @echo "$(COLOR_BOLD)$(COLOR_BLUE)===== $(COLOR_RESET)"
    @echo "$(COLOR_CYAN) AWS Load Balancer Controller creates ALBs from Ingress resources$(COLOR_RESET)"
    @echo "$(COLOR_CYAN) We wait until ALBs have a public DNS hostname$(COLOR_RESET)"
    @echo ""
    @echo "$(COLOR_BLUE)► Waiting for task-manager ALB...$(COLOR_RESET)"
    @START=$(date +%s); \
    for i in $(seq 1 30); do \
        NOW=$(date +%s); \
        ELAPSED=$((NOW - START)); \
        HOST=$(kubectl get ingress -n $(NAMESPACE) \
            -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}' 2>/dev/null); \
        if [ -n "$$HOST" ]; then \
            echo "$(COLOR_GREEN) ✓ Task-manager ALB ready after $$ELAPSEDs$(COLOR_RESET)"; \
            echo " $(COLOR_GRAY)→ DNS: $$HOST$(COLOR_RESET)"; \
            break; \
        fi; \
        echo " [$$ELAPSED] Attempt $$i/30: ALB not yet provisioned, waiting 10s..."; \
        sleep 10; \
    done
    @echo ""
    @echo "$(COLOR_BLUE)► Waiting for Grafana ALB...$(COLOR_RESET)"
    @START=$(date +%s); \
    for i in $(seq 1 30); do \
        NOW=$(date +%s); \
        ELAPSED=$((NOW - START)); \
        HOST=$(kubectl get ingress -n monitoring \
            -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}' 2>/dev/null); \
        if [ -n "$$HOST" ]; then \
            echo "$(COLOR_GREEN) ✓ Grafana ALB ready after $$ELAPSEDs$(COLOR_RESET)"; \
            echo " $(COLOR_GRAY)→ DNS: $$HOST$(COLOR_RESET)"; \
            break; \
        fi; \
        echo " [$$ELAPSED] Attempt $$i/30: ALB not yet provisioned, waiting 10s..."; \
        sleep 10; \
    done

```

Hình 80. Stage 5 makefile wait-alb

Makefile cũng chờ ALB của Grafana trong namespace monitoring. Khi ALB đã có hostname, stage DNS phase 2 mới chạy:

Biến alb_exists=true báo cho Terraform DNS state biết rằng ALB đã tồn tại, lúc này Terraform có thể lookup ALB và tạo Route53 alias record. Cách này xử lý đúng vấn đề phụ thuộc vòng: DNS cần ALB, nhưng ALB chỉ được tạo sau khi Kubernetes Ingress được apply.

3.7.10 Kiểm tra trạng thái bằng make status

Target make status dùng để xem nhanh trạng thái hiện tại của hệ thống.

```
.PHONY: status
status: ## Check trạng thái infrastructure

@echo ""
@echo "$(COLOR_BLUE)$(COLOR_BOLD)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE)$(COLOR_BOLD) Infrastructure Status$(COLOR_RESET)"
@echo "$(COLOR_BLUE)$(COLOR_BOLD)===== $(COLOR_RESET)"
@echo ""
@echo "$(COLOR_BLUE)» EKS Cluster$(COLOR_RESET)"
@aws eks describe-cluster --name $(CLUSTER_NAME) --region $(REGION) \
--query 'cluster.{Status:status,Version:version,Endpoint:endpoint}' \
--output table 2>/dev/null || echo " X Cluster not found"
@echo ""
@echo "$(COLOR_BLUE)» EKS Node Groups$(COLOR_RESET)"
@aws eks list-nodegroups --cluster-name $(CLUSTER_NAME) --region $(REGION) \
--query 'nodegroups[]' --output text 2>/dev/null | while read ng; do \
if [ -n "$$ng" ]; then \
aws eks describe-nodegroup --cluster-name $(CLUSTER_NAME) \
--nodegroup-name $$ng --region $(REGION) \
--query 'nodegroup.{Name:nodegroupName,Status:status,Desired:scalingConfig.desiredSize}' \
--output table; \
fi; \
done
@echo ""
@echo "$(COLOR_BLUE)» RDS$(COLOR_RESET)"
@aws rds describe-db-instances --region $(REGION) \
--query 'DBInstances[?contains(DBInstanceIdentifier, `devops`)].{ID:DBInstanceIdentifier,Status:DBInstanceStatus,Class:DBInstanceClass}' \
--output table 2>/dev/null || echo " X No RDS"
@echo ""
@echo "$(COLOR_BLUE)» ArgoCD Applications$(COLOR_RESET)"
@kubectl get applications -n $(ARGOCD_NAMESPACE) 2>/dev/null || echo " X Cannot connect"
@echo ""
@echo "$(COLOR_BLUE)» Workload Pods (task-manager-dev)$(COLOR_RESET)"
@kubectl get pods -n $(NAMESPACE) --no-headers 2>/dev/null | \
awk '{print " " $$1 ": " $$3}' || echo " X Cannot connect"
@echo ""
@echo "$(COLOR_BLUE)» Monitoring Pods$(COLOR_RESET)"
@kubectl get pods -n monitoring --no-headers 2>/dev/null | head -8 | \
awk '{print " " $$1 ": " $$3}' || echo " X Cannot connect"
```

Hình 81. Makefile status

Ngoài trạng thái cluster và workload, Makefile còn kiểm tra các tài nguyên đang phát sinh chi phí. Điều này phù hợp với môi trường đồ án/dev vì người vận hành cần biết hiện tại còn bao nhiêu NAT Gateway hoặc Load Balancer đang chạy để kiểm soát chi phí.

3.7.11 Verify end-to-end sau triển khai

Target make verify dùng để kiểm tra hệ thống sau deploy.

```
.PHONY: verify
verify: ## Verify end-to-end

@echo ""
@echo "$(COLOR_BLUE)$(COLOR_BOLD)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE)$(COLOR_BOLD) End-to-End Verification$(COLOR_RESET)"
@echo "$(COLOR_BLUE)$(COLOR_BOLD)===== $(COLOR_RESET)"
@echo ""
@echo "$(COLOR_BLUE)» ExternalSecrets sync status$(COLOR_RESET)"
@kubectl get externalsecret -A 2>/dev/null | head -10 || echo " X Cannot get"
@echo ""
@echo "$(COLOR_BLUE)» App health check$(COLOR_RESET)"
@DOMAIN=$(cd $(ENVSDIR)/dns && terraform output -raw full_fqdn 2>/dev/null); \
if [ -n "$$DOMAIN" ]; then \
echo " $(COLOR_GRAY)» URL: https://$$DOMAIN/api/health/ready$(COLOR_RESET)"; \
for i in $(seq 1 12); do \
STATUS=$(curl -sk -o /dev/null -w "%{http_code}" https://$$DOMAIN/api/health/ready 2>/dev/null); \
if [ "$$STATUS" = "200" ]; then \
echo "$(COLOR_GREEN) ✓ HTTP 200 OK$(COLOR_RESET)"; break; \
fi; \
echo " [Attempt $$i/12] HTTP $$STATUS, retry in 10s..."; sleep 10; \
done; \
fi
@echo ""
@echo "$(COLOR_BLUE)» Grafana endpoint$(COLOR_RESET)"
@curl -sk -o /dev/null -w " $(COLOR_GRAY)» grafana.cookieal.site$(COLOR_RESET) → HTTP %{http_code}\n" \
https://grafana.cookieal.site/api/health 2>/dev/null
@echo ""
@echo "$(COLOR_BLUE)» Grafana credentials$(COLOR_RESET)"
@PASS=$(kubectl get secret grafana-admin-secret -n monitoring \
-o jsonpath='{.data.admin-password}' 2>/dev/null | base64 -d); \
if [ -n "$$PASS" ]; then \
echo " Username: admin"; \
echo " Password: $$PASS"; \
else \
echo " ⚠ Cannot retrieve credentials"; \
fi
@echo ""
```

Hình 82. Makefile verify


```
.PHONY: wake
wake: ## 🟡 Bật lại hệ thống (sau đó chạy make wake-dns)

@echo ""
@echo "$(COLOR_BLUE)$(COLOR_BOLD)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE)$(COLOR_BOLD) 🟡 WAKE UP: resume hibernated system $(COLOR_RESET)"
@echo "$(COLOR_BLUE)$(COLOR_BOLD)===== $(COLOR_RESET)"
@echo ""
@echo "$(COLOR_BLUE)▶ [1/3] Starting RDS (~5-7 min)...$(COLOR_RESET)"
@DB_ID=$(cd $(ENV_DIR)/rds && terraform output -raw db_endpoint 2>/dev/null | cut -d'.' -f1); \
if [ -n "$$DB_ID" ]; then \
  STATUS=$(aws rds describe-db-instances --db-instance-identifier $$DB_ID \
    --region $(REGION) --query 'DBInstances[0].DBInstanceStatus' --output text 2>/dev/null); \
  echo " $(COLOR_GRAY)→ Current state: $$STATUS$(COLOR_RESET)"; \
  if [ "$$STATUS" = "stopped" ]; then \
    aws rds start-db-instance --db-instance-identifier $$DB_ID --region $(REGION) > /dev/null; \
    echo " $(COLOR_GRAY)→ Waiting for RDS available...$(COLOR_RESET)"; \
    aws rds wait db-instance-available --db-instance-identifier $$DB_ID --region $(REGION); \
    echo "$(COLOR_GREEN) ✓ RDS available$(COLOR_RESET)"; \
  fi; \
fi
@echo "$(COLOR_BLUE)▶ [2/3] Scaling EKS node group up...$(COLOR_RESET)"
@NODEGROUP=$(aws eks list-nodegroups --cluster-name $(CLUSTER_NAME) \
  --region $(REGION) --query 'nodegroups[0]' --output text 2>/dev/null); \
if [ -n "$$NODEGROUP" ] && [ "$$NODEGROUP" != "None" ]; then \
  aws eks update-nodegroup-config --cluster-name $(CLUSTER_NAME) \
    --nodegroup-name $$NODEGROUP \
    --scaling-config minSize=1,maxSize=3,desiredSize=2 \
    --region $(REGION) > /dev/null; \
  echo " $(COLOR_GRAY)→ Waiting for nodes Ready (~3 min)...$(COLOR_RESET)"; \
  for i in $(seq 1 18); do \
    READY=$(kubectl get nodes --no-headers 2>/dev/null | grep -c " Ready" || echo "0"); \
    if [ "$$READY" -ge "1" ]; then \
      echo "$(COLOR_GREEN) ✓ $$READY nodes Ready$(COLOR_RESET)"; break; \
    fi; \
    echo " [$(i*10)s] $$READY nodes Ready, waiting..."; sleep 10; \
  done; \
fi
@echo "$(COLOR_BLUE)▶ [3/3] Triggering ArgoCD sync (recreates Ingress → new ALBs)...$(COLOR_RESET)"
@kubectl patch application task-manager-dev -n argocd \
  --type merge -p '{"operation":{"sync":{"prune":true,"revision":"HEAD"}}}' 2>/dev/null || true
```

Hình 84. Makefile wake

Sau khi wake, ALB mới có thể có DNS name mới, nên Makefile yêu cầu chạy make wake-dns. Target này gọi lại `_stage-5-wait-alb`, `_stage-6-dns-phase2`. Nhờ đó Route53 được cập nhật để trỏ về ALB mới.

3.7.12 Các lệnh hỗ trợ vận hành hằng ngày

Makefile còn có một số target tiện ích:

`argocd-ui`:

```
.PHONY: argocd-ui
argocd-ui: ## Port-forward ArgoCD UI tới http://localhost:8080

@echo ""
@echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
@echo "$(COLOR_BLUE) ArgoCD UI Access$(COLOR_RESET)"
@echo "$(COLOR_BLUE)===== $(COLOR_RESET)"
@echo "$(COLOR_CYAN)URL: https://localhost:8080 (ignore SSL warning)$(COLOR_RESET)"
@echo "$(COLOR_CYAN)Username: admin$(COLOR_RESET)"
@echo "$(COLOR_CYAN>Password:$(COLOR_RESET)"
@kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" 2>/dev/null | base64 -d
@echo ""
@echo ""
@echo "$(COLOR_YELLOW)Press Ctrl+C to stop port-forward$(COLOR_RESET)"
@echo ""
@kubectl port-forward -n argocd svc/argocd-server 8080:443
```

Hình 85. Makefile argocd-ui

```

.PHONY: logs-backend
logs-backend: ## Tail backend logs
    @kubectl logs -n $(NAMESPACE) -l app=backend --tail=50 -f

.PHONY: logs-frontend
logs-frontend: ## Tail frontend logs
    @kubectl logs -n $(NAMESPACE) -l app=frontend --tail=50 -f

.PHONY: shell-backend
shell-backend: ## Mở shell vào 1 backend pod
    @POD=$(kubectl get pod -n $(NAMESPACE) -l app=backend -o jsonpath='{.items[0].metadata.name}'); \
    kubectl exec -it -n $(NAMESPACE) $$POD -- sh

```

Hình 86. Makefile logs-backend, logs-frontend, shell-backend

Các lệnh này hỗ trợ quan sát ArgoCD UI, xem log backend/frontend và truy cập shell vào backend Pod khi cần debug. Nhờ gom vào Makefile, người vận hành không cần nhớ selector, namespace hoặc service name mỗi lần thao tác.

Chương 4. KẾT QUẢ & KẾT LUẬN

4.1 Kết quả đạt được

Sau quá trình thực hiện, nhóm đã xây dựng được một hệ thống Task Manager triển khai theo hướng DevOps/GitOps end-to-end. Ứng dụng có frontend React/Vite, backend Node.js Express, database PostgreSQL/RDS, xác thực JWT, health check, logging, metrics và migration script. Backend và frontend được đóng gói thành Docker image riêng bằng multi-stage build và chạy bằng non-root user.

Pipeline GitHub Actions đã tự động hóa quá trình lint, test, scan dependency, scan secret, phân tích chất lượng mã nguồn, build image, scan image, push Docker Hub và cập nhật config repo. Terraform đã mô tả được hạ tầng AWS theo nhiều state gồm network, EKS, RDS, secrets và DNS. Config repo đã chứa Helm chart, ArgoCD applications, monitoring rules và dashboard. ArgoCD đã đảm nhận vai trò đồng bộ GitOps. Argo Rollouts đã hỗ trợ blue/green deployment cho backend. External Secrets đã tách secret khỏi Git. Prometheus/Grafana đã cung cấp dashboard theo RED method.

Nhóm kết quả	Nội dung hoàn thành
Ứng dụng	Task Manager full-stack với auth, task CRUD, health, metrics
Container	Dockerfile backend/frontend, multi-stage, non-root runtime
CI/CD	GitHub Actions test, scan, build, push, GitOps promotion
IaC	Terraform VPC, EKS, RDS, Secrets, DNS/ACM
GitOps	Config repo, Helm chart, ArgoCD app-of-apps
Delivery	Argo Rollouts blue/green cho backend
Security	External Secrets, NetworkPolicy, scanning, securityContext
Observability	Prometheus metrics, ServiceMonitor, PrometheusRule, Grafana
Operations	Makefile deploy/destroy/hibernate/wake/verify

Bảng 3. Các kết quả đạt được trong sau quá trình thực hiện

4.2 Đánh giá hệ thống

Về mức độ tự động hóa, hệ thống đã giảm đáng kể thao tác thủ công. Developer push code, CI xử lý build/scan và ArgoCD sync cluster. Về khả năng tái tạo, Terraform và Helm giúp hạ tầng và manifest có thể dựng lại từ code. Về bảo mật, hệ thống có nhiều lớp kiểm soát từ code, image, secret đến runtime. Về quan sát, backend metrics và Grafana dashboard giúp theo dõi các chỉ số RED cơ bản. Về rollback, GitOps cho phép

rollback bằng Git commit, còn Argo Rollouts giúp kiểm tra phiên bản backend trước khi chuyển traffic.

Tuy nhiên, đây vẫn là môi trường dev/thesis nên một số cấu hình chưa bật ở mức production. Replica count thấp để tiết kiệm chi phí, HPA/PDB chưa bật trong AWS dev, một số scan đang soft-fail để phục vụ quá trình phát triển, migration còn đơn giản và chưa có load test đầy đủ để chứng minh ngưỡng chịu tải.

4.3 Hạn chế

Các hạn chế chính của đồ án gồm:

1. Ứng dụng nghiệp vụ đơn giản, chủ yếu phục vụ mục tiêu minh họa DevOps.
2. Môi trường AWS dev chưa bật đầy đủ autoscaling, PDB và multi-replica cho mọi component.
3. Canary deployment và Prometheus-based analysis mới ở mức thiết kế/mở rộng, chưa có traffic preview đủ lớn để đánh giá production.
4. Migration dùng `sequelize.sync()` đơn giản, chưa có migration versioning chuyên nghiệp như Sequelize CLI, Flyway hoặc Liquibase.
5. Chưa có load test/k6 chính thức để đo throughput, latency và khả năng chịu tải.
6. Chưa có alerting SLO đầy đủ hoặc incident response playbook.
7. Chi phí AWS là yếu tố cần kiểm soát, nên nhiều cấu hình được tối ưu cho dev thay vì high availability.

4.4 Hướng phát triển

Trong tương lai, hệ thống có thể phát triển theo các hướng sau:

1. Bật HPA và PodDisruptionBudget cho môi trường staging/prod.
2. Hoàn thiện canary deployment với Argo Rollouts và Prometheus queries dựa trên traffic thật.
3. Chuyển migration sang công cụ có version tracking và rollback rõ ràng.
4. Bổ sung load test bằng k6 và đưa kết quả vào dashboard.
5. Xây dựng alerting theo SLO: error budget, p95/p99 latency, availability.
6. Tăng cường bảo mật bằng WAF, CloudFront, private registry policy, OPA/Gatekeeper hoặc Kyverno.
7. Thiết kế backup/restore drill cho RDS và disaster recovery cho cluster.
8. Tách Terraform repo riêng hoặc module hóa sâu hơn để hỗ trợ nhiều môi trường.
9. Tích hợp notification cho CI/CD và ArgoCD qua Slack, email hoặc webhook.

4.5 Kết luận

Đồ án đã chứng minh được một quy trình DevOps/GitOps hoàn chỉnh cho một ứng dụng web ba tầng. Từ source code, hệ thống được kiểm thử, scan, đóng gói thành Docker image, push lên registry, cập nhật config repo và triển khai lên AWS EKS thông qua ArgoCD. Trong quá trình vận hành, hệ thống có quản lý secret bằng AWS Secrets Manager, progressive delivery bằng Argo Rollouts, observability bằng Prometheus/Grafana và automation bằng Makefile/Terraform.

Thông qua đồ án, nhóm hiểu rõ hơn rằng triển khai phần mềm hiện đại không chỉ là đưa ứng dụng chạy được, mà là xây dựng một quy trình có thể lặp lại, có thể kiểm soát và có khả năng phản hồi khi có sự cố. Những thành phần như CI/CD, IaC, GitOps,

Kubernetes, monitoring và security không tách rời nhau; chúng phối hợp để tạo nên một hệ thống đáng tin cậy hơn. Đây là nền tảng quan trọng để tiếp tục phát triển các hệ thống cloud-native ở quy mô lớn hơn.

TÀI LIỆU THAM KHẢO

- [1] GitHub, “GitHub Actions documentation,” *GitHub Docs*. [Online]. Available: <https://docs.github.com/en/actions>. Accessed: Jun. 8, 2026.
- [2] HashiCorp, “Terraform Documentation,” *HashiCorp Developer*. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>. Accessed: Jun. 8, 2026.
- [3] Amazon Web Services, “What is Amazon EKS?,” *Amazon EKS User Guide*. [Online]. Available: <https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html>. Accessed: Jun. 8, 2026.
- [4] Kubernetes, “Kubernetes Concepts,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/>. Accessed: Jun. 8, 2026.
- [5] Helm Authors, “Helm Documentation,” *Helm Docs*. [Online]. Available: <https://helm.sh/docs/>. Accessed: Jun. 8, 2026.
- [6] Argo Project, “Argo CD - Declarative GitOps CD for Kubernetes,” *Argo CD Documentation*. [Online]. Available: <https://argo-cd.readthedocs.io/en/stable/>. Accessed: Jun. 8, 2026.
- [7] Argo Project, “Argo Rollouts,” *Argo Rollouts Documentation*. [Online]. Available: <https://argoproj.github.io/rollouts/>. Accessed: Jun. 8, 2026.
- [8] External Secrets Operator, “Introduction,” *External Secrets Operator Documentation*. [Online]. Available: <https://external-secrets.io/>. Accessed: Jun. 8, 2026.
- [9] Docker, “Build, tag, and publish an image,” *Docker Docs*. [Online]. Available: <https://docs.docker.com/get-started/docker-concepts/building-images/build-tag-and-publish-an-image/>. Accessed: Jun. 8, 2026.
- [10] Aqua Security, “Trivy,” *Trivy Documentation*. [Online]. Available: <https://trivy.dev/>. Accessed: Jun. 8, 2026.
- [11] SonarSource, “Understanding quality gates,” *SonarQube Server Documentation*. [Online]. Available: <https://docs.sonarsource.com/sonarqube/latest/user-guide/quality-gates/>. Accessed: Jun. 8, 2026.
- [12] Gitleaks, “Gitleaks,” *GitHub Repository*. [Online]. Available: <https://github.com/gitleaks/gitleaks>. Accessed: Jun. 8, 2026.
- [13] Prometheus Authors, “Overview,” *Prometheus Documentation*. [Online]. Available: <https://prometheus.io/docs/>. Accessed: Jun. 8, 2026.

- [14] Grafana Labs, “Grafana documentation,” *Grafana Documentation*. [Online]. Available: <https://grafana.com/docs/>. Accessed: Jun. 8, 2026.
- [15] Amazon Web Services, “Amazon RDS for PostgreSQL,” *Amazon RDS User Guide*. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html. Accessed: Jun. 8, 2026.
- [16] Amazon Web Services, “AWS Secrets Manager Documentation,” *AWS Documentation*. [Online]. Available: <https://docs.aws.amazon.com/secretsmanager/>. Accessed: Jun. 8, 2026.
- [17] Amazon Web Services, “AWS Certificate Manager Documentation,” *AWS Documentation*. [Online]. Available: <https://docs.aws.amazon.com/acm/>. Accessed: Jun. 8, 2026.
- [18] Amazon Web Services, “Amazon Route 53 Documentation,” *AWS Documentation*. [Online]. Available: <https://aws.amazon.com/documentation-overview/route53/>. Accessed: Jun. 8, 2026.
- [19] Kubernetes SIGs, “AWS Load Balancer Controller,” *AWS Load Balancer Controller Documentation*. [Online]. Available: <https://kubernetes-sigs.github.io/aws-load-balancer-controller/>. Accessed: Jun. 8, 2026.
- [20] Kubernetes, “Kustomize,” *Kubernetes Documentation*. [Online]. Available: <https://kubectl.docs.kubernetes.io/references/kustomize/>. Accessed: Jun. 8, 2026.